

Database Overview



You said:

What is a Database? is a collection of data in a format that can be easily accessed. Why databases? • can store large data • features like security, scalability etc. • Easier to insert, update or delete data

ChatGPT said:

Your definition is **basically correct**, but let me refine it so it sounds stronger (especially for interviews 🙌):

📌 What is a Database?

A **Database** is an organized collection of data that is stored and managed in a way that allows easy access, retrieval, updating, and management.

👉 In simple words:

A database stores data in a structured format so we can easily use it when needed.

📌 Why do we use Databases?

Here are the key reasons (with better explanation):

1. Store Large Amount of Data

- Databases can handle huge volumes of data efficiently.

2. Data Security 🛡️

- Provide authentication and authorization to protect data.

3. Scalability 📈

- Can grow as your application grows (more users, more data).

4. Easy Data Management

- Insert, update, delete data easily using queries.

5. Data Consistency & Integrity

- Ensures data remains accurate and reliable.

6. Multi-user Access

- Multiple users can access data at the same time.

📌 Example

- Student records system
- Banking system
- E-commerce websites (like storing products, users, orders)

📌 Interview Answer (Short Version)

👉 "A database is an organized collection of data that allows efficient storage, retrieval, and management of information. It helps handle large data with features like security, scalability, and easy data operations."

If you want, I can next teach you:

- ✓ Types of Databases (SQL vs NoSQL)
- ✓ DBMS vs Database
- ✓ Real interview questions on this topic

You said:

sql vs nosql

ChatGPT said:

Here’s a **clear, interview-ready comparison of SQL vs NoSQL** 📌

📌 **SQL vs NoSQL**

Feature	SQL (Relational DB)	NoSQL (Non-Relational DB)
 Structure	Tables (rows & columns)	Flexible (JSON, key-value, graph, etc.)
 Schema	Fixed schema (predefined)	Dynamic schema (flexible)
 Relationships	Strong relationships (JOINS)	Weak or no relationships
 Scalability	Vertical (increase server power)	Horizontal (add more servers)
 Query Language	Structured Query Language (SQL)	No fixed query language
 Data Type	Structured data only	Structured + Unstructured
 ACID Properties	Fully supported	Sometimes (depends on DB)
 Performance	Best for complex queries	Best for large-scale distributed data

📌 **Examples**

SQL Databases

- MySQL
- PostgreSQL
- Oracle Database

NoSQL Databases

- MongoDB

- Cassandra
- Redis

📌 When to Use SQL?

- ✓ When data is structured
- ✓ When relationships are important (e.g., banking system)
- ✓ When you need strong consistency (ACID)

📌 When to Use NoSQL?

- ✓ When data is unstructured or semi-structured
- ✓ When handling big data or real-time apps
- ✓ When scalability is important (e.g., social media)

📌 Real-Life Example

- **Banking System** → SQL (accuracy + relationships)
- **Instagram / Netflix** → NoSQL (huge, fast, scalable data)

📌 Interview Answer (Best Short Version)

👉 *"SQL databases use structured tables with a fixed schema and are best for relational data and complex queries. NoSQL databases use flexible schemas and are designed for scalability and handling large, unstructured data."*

If you want next, I can give:

- ✓ SQL commands (SELECT, INSERT, UPDATE, DELETE) with examples
- ✓ 20 interview questions on SQL vs NoSQL
- ✓ Mini project using SQL 🚀

You said:

what is sql explain

ChatGPT said:

📌 What is SQL?

SQL (Structured Query Language) is a programming language used to **communicate with databases**.

It is used to **store, retrieve, update, and delete data** in a database.

👉 In simple words:

SQL is the language we use to talk to a database.

📌 Where is SQL Used?

SQL is used in relational database systems like:

- MySQL
- PostgreSQL
- Oracle Database

📌 What Can You Do with SQL?

1. **Insert Data** → Add new records
2. **Retrieve Data** → Get data (using SELECT)
3. **Update Data** → Modify existing data
4. **Delete Data** → Remove records
5. **Create Tables/Databases**

📌 Basic SQL Commands (with Example)

1. SELECT (Read Data)

```
SQL
SELECT * FROM students;
```

2. INSERT (Add Data)

```
SQL
INSERT INTO students (name, age) VALUES ('Aamir', 22);
```

3. UPDATE (Modify Data)

```
SQL
UPDATE students SET age = 23 WHERE name = 'Aamir';
```

4. DELETE (Remove Data)

```
SQL
DELETE FROM students WHERE name = 'Aamir';
```

Types of SQL Commands

1. DDL (Data Definition Language)

- CREATE, ALTER, DROP

2. DML (Data Manipulation Language)

- INSERT, UPDATE, DELETE

3. DQL (Data Query Language)

- SELECT

4. DCL (Data Control Language)

- GRANT, REVOKE

Why is SQL Important?

- ✓ Easy to learn
- ✓ Used in almost every company
- ✓ Essential for Data Analytics, Backend, and Data Science
- ✓ Works with large data efficiently

Interview Answer (Best Short Version)

👉 *"SQL is a standard language used to interact with relational databases. It allows us to store, retrieve, update, and manage data efficiently using commands like SELECT, INSERT, UPDATE, and DELETE."*

If you want next, I can give:

- 🔥 20 SQL interview questions
- 🔥 Real-world SQL project
- 🔥 Practice questions with solutions

You said:

what is table

ChatGPT said:

What is a Table in SQL?

A **table** is a structure in a database that stores data in the form of **rows and columns**.

👉 In simple words:

A table is like an **Excel sheet**, where:

- **Columns** = fields (name, age, etc.)
- **Rows** = records (actual data)

Example of a Table

👉 Student Table

ID	Name	Age
----	------	-----

1	Aamir	22
---	-------	----

2	Rahul	21
---	-------	----

3	Sara	23
---	------	----

- **Columns** → ID, Name, Age
- **Rows** → Each student's data

Key Points

- A database contains **multiple tables**
- Each table stores **related data**
- Tables are connected using **keys** (like Primary Key, Foreign Key)

Example SQL to Create Table

```
SQL
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT
);
```

📌 Interview Answer (Short)

👉 "A table is a collection of data organized in rows and columns within a database. Each row represents a record, and each column represents a field."

If you want, next I can explain:

- ✅ Primary Key & Foreign Key (very important)
- ✅ How tables are connected
- ✅ Real interview questions on tables

You said:

tuple means easy way

ChatGPT said:

📌 Tuple (in Database) – Easy Way

👉 A **tuple** means a **single row (one record) in a table**.

📌 Simple Understanding

Think of a table like Excel:

ID	Name	Age
----	------	-----

1	Aamir	22
---	-------	----

2	Rahul	21
---	-------	----

👉 Each **row = 1 tuple**

📌 One Line Definition (Interview)

👉 "A tuple is a single row in a database table that represents one record."

📌 Super Easy Trick to Remember

- **Table** = full data
- **Column** = field

- **Tuple** = one row

If you want, I can next explain:

- ✓ Attribute (column)
- ✓ Primary Key vs Foreign Key
- ✓ Real SQL interview questions 🚀

You said:

Explain Attribute (column)

ChatGPT said:

📌 What is an Attribute (Column)?

An **attribute** (also called a **column**) is a **field in a table that stores a specific type of information** about each record.

👉 In simple words:

Column = what type of data you store

📌 Example

👉 Student Table

ID Name Age

- **ID** → attribute
- **Name** → attribute
- **Age** → attribute

👉 These columns define **what information each student record will have**

📌 Key Points

- Each column has a **name** (like Name, Age)
- Each column has a **data type** (INT, VARCHAR, etc.)

- All values in a column are of the **same type**

📌 Example in SQL

```
SQL
CREATE TABLE students (
    id INT,
    name VARCHAR(50),
    age INT
);
```

👉 Here:

- **id, name, age** = **attributes (columns)**

📌 Interview Answer (Short)

👉 "An attribute is a column in a database table that represents a specific property of an entity and stores a particular type of data."

📌 Easy Comparison

- **Table** → Full data
- **Row (Tuple)** → One record
- **Column (Attribute)** → One field

If you want next, I can explain:

- ✓ Primary Key (very important 🔥)
- ✓ Foreign Key with examples
- ✓ Practice questions for interview 🚀

You said:

```
CREATE DATABASE college; create database xyz_company; DROP DATABASE
xyz_company; USE college; CREATE TABLE student( rollno INT, NAME
varchar(30), age INT ); INSERT INTO student VALUES (101, "ADAM", 22), (102,
"BOB", 21); SELECT * FROM student; CREATE DATABASE IF NOT EXISTS student;
CREATE DATABASE IF NOT EXISTS college; DROP DATABASE IF EXISTS college;
SHOW DATABASES; CREATE DATABASE IF NOT EXISTS instagram; USE
instagram; SHOW TABLES; DROP TABLE IF EXISTS user; CREATE TABLE user( id
INT, age INT, name VARCHAR(30) NOT NULL, email VARCHAR(50) UNIQUE,
followers INT DEFAULT 0, following INT, CONSTRAINT CHECK (age>=13),
```

```
PRIMARY KEY (id) ); CREATE TABLE post( id INT PRIMARY KEY, content
VARCHAR(100), user_id INT, FOREIGN KEY (user_id) REFERENCES user(id) );
INSERT INTO user (id, age, name, email, followers, following) VALUES (1, 22,
"adam", "adam123@gmail.com", 455, 12), (2, 21, "kamran",
"kamran223@gmail.com", 12345, 34), (3, 19, "rahul", "rahul123@gmail.com",
566, 56), (4, 22, "vikash", "vikash123@gmail.com", 122, 896); SELECT * FROM
user; SELECT id, name, email FROM user; SELECT id, name, age FROM user;
SELECT name, email FROM user; SELECT id, name, email FROM user; SELECT
DISTINCT age FROM user; explain all query
```

ChatGPT said:

You've written a solid set of SQL queries 👍 — now let's **explain each part in a simple + interview-ready way.**

1. Database Creation & Deletion

Create Database

```
SQL
CREATE DATABASE college;
create database xyz_company;
```

✓ Creates new databases named **college** and **xyz_company**

Delete Database

```
SQL
DROP DATABASE xyz_company;
```

✓ Deletes the database permanently ⚠️ (all data lost)

Use Database

```
SQL
USE college;
```

✓ Selects the database you want to work with

2. Create Table

```
SQL
CREATE TABLE student (
  rollno INT,
  NAME varchar(30),
  age INT
);
```

- ✓ Creates a **table** named **student**
- ✓ Columns:
 - rollno → integer
 - name → text (max 30 characters)
 - age → integer

3. Insert Data

```
SQL
INSERT INTO student VALUES
(101, "ADAM", 22),
(102, "BOB", 21);
```

- ✓ Adds **rows (tuples)** into the table
- ✓ Each row = one student record

4. Fetch Data

```
SQL
SELECT * FROM student;
```

- ✓ * means → get **all columns and all rows**

5. Conditional Database Creation

```
SQL
CREATE DATABASE IF NOT EXISTS student;
CREATE DATABASE IF NOT EXISTS college;
```


- ✓ Creates database **only if it doesn't already exist**
- ✓ Prevents errors

6. Safe Delete Database

```
SQL
DROP DATABASE IF EXISTS college;
```

- ✓ Deletes database only if it exists
- ✓ Avoids error

7. Show Databases

```
SQL
SHOW DATABASES;
```

- ✓ Displays all databases in your system

8. Instagram Database

```
SQL
CREATE DATABASE IF NOT EXISTS instagram;
USE instagram;
SHOW TABLES;
```

- ✓ Creates & selects database
- ✓ **SHOW TABLES** → shows tables inside database

9. Drop Table

```
SQL
DROP TABLE IF EXISTS user;
```

- ✓ Deletes table safely

10. Create **user** Table (Important)

```
SQL
CREATE TABLE user(
  id INT,
  age INT,
  name VARCHAR(30) NOT NULL,
  email VARCHAR(50) UNIQUE,
  followers INT DEFAULT 0,
  following INT,
  CONSTRAINT CHECK (age>=13),
  PRIMARY KEY (id)
);
```

Explanation:

- **id INT** → user ID
- **name NOT NULL** → cannot be empty
- **email UNIQUE** → no duplicate emails allowed
- **followers DEFAULT 0** → default value is 0
- **CHECK (age>=13)** → age must be ≥ 13
- **PRIMARY KEY (id)** → unique + not null identifier

11. Create **post** Table (Foreign Key)

```
SQL
CREATE TABLE post(
  id INT PRIMARY KEY,
  content VARCHAR(100),
  user_id INT,
  FOREIGN KEY (user_id) REFERENCES user(id)
);
```

- ✓ **user_id** links to **user** table
- ✓ This creates a **relationship between tables**

12. Insert Data into user

```
SQL
INSERT INTO user
(id, age, name, email, followers, following)
VALUES
(1, 22, "adam", "adam123@gmail.com", 455, 12),
(2, 21, "kamran", "kamran223@gmail.com", 12345, 34),
(3, 19, "rahul", "rahul123@gmail.com", 566, 56),
(4, 22, "vikash", "vikash123@gmail.com", 122, 896);
```

- ✓ Inserts multiple rows at once

13. SELECT Queries

Get All Data

```
SQL
SELECT * FROM user;
```

- ✓ Returns all columns

Specific Columns

```
SQL
SELECT id, name, email FROM user;
```

- ✓ Only selected columns shown

```
SQL
SELECT id, name, age FROM user;
```

- ✓ Shows id, name, age

```
SQL
SELECT name, email FROM user;
```

- ✓ Only name and email

14. DISTINCT (Important)

```
SQL
SELECT DISTINCT age FROM user;
```

- ✓ Removes duplicate values
- ✓ Shows **unique ages only**

 Example:

If ages = 22, 21, 19, 22

Output = 22, 21, 19

Final Interview Summary

 "These queries demonstrate database creation, table creation, inserting data, applying constraints like PRIMARY KEY, UNIQUE, CHECK, and retrieving data using SELECT and DISTINCT. They also show relationships using FOREIGN KEY"

If you want next level 

I can teach you:

- ✓ WHERE clause (filter data)
- ✓ ORDER BY, LIMIT
- ✓ JOIN (most important for interviews)
- ✓ 30 SQL interview questions with answers 

You said:

create table from scratch to advanced query for learning

ChatGPT said:

Great — you're ready to go from **basic** → **advanced SQL in one flow**. I'll build a **complete learning script** step by step so you can practice like a real project



1. Create Database & Use It

```
SQL
CREATE DATABASE IF NOT EXISTS company_db;
USE company_db;
```

2. Create Tables (with Constraints)

Employees Table

```
SQL
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT CHECK (age >= 18),
    department VARCHAR(50),
    salary INT DEFAULT 0
);
```

Departments Table

```
SQL
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) UNIQUE
);
```

Projects Table (Foreign Key)

```
SQL
CREATE TABLE projects (
    project_id INT PRIMARY KEY,
    project_name VARCHAR(100),
    emp_id INT,
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id)
);
```

3. Insert Data

```
SQL
INSERT INTO employees VALUES
(1, 'Aamir', 22, 'IT', 50000),
(2, 'Rahul', 25, 'HR', 40000),
(3, 'Sara', 23, 'IT', 60000),
(4, 'John', 30, 'Finance', 70000);

INSERT INTO departments VALUES
(101, 'IT'),
(102, 'HR'),
(103, 'Finance');

INSERT INTO projects VALUES
(201, 'Website', 1),
(202, 'App', 3),
(203, 'Audit', 4);
```

4. Basic Queries

Get all data

```
SQL
SELECT * FROM employees;
```

Specific columns

```
SQL
SELECT name, salary FROM employees;
```

5. WHERE Clause (Filtering)

```
SQL
SELECT * FROM employees WHERE salary > 50000;
SELECT * FROM employees WHERE department = 'IT';
```

6. ORDER BY (Sorting)

```
SQL
SELECT * FROM employees ORDER BY salary DESC;
SELECT * FROM employees ORDER BY age ASC;
```

7. LIMIT (Top Records)

```
SQL
SELECT * FROM employees LIMIT 2;
```

8. Aggregate Functions 🔥

```
SQL
SELECT COUNT(*) FROM employees;
SELECT MAX(salary) FROM employees;
SELECT MIN(salary) FROM employees;
SELECT AVG(salary) FROM employees;
```

9. GROUP BY (Very Important 🔥)

```
SQL
SELECT department, COUNT(*)
FROM employees
GROUP BY department;
```

10. HAVING (Filter Groups)

```
SQL
SELECT department, AVG(salary)
FROM employees
GROUP BY department
HAVING AVG(salary) > 45000;
```

11. JOIN (Most Important)

INNER JOIN

```
SQL
SELECT e.name, p.project_name
FROM employees e
INNER JOIN projects p
ON e.emp_id = p.emp_id;
```

LEFT JOIN

```
SQL
SELECT e.name, p.project_name
FROM employees e
LEFT JOIN projects p
ON e.emp_id = p.emp_id;
```

12. UPDATE Data

```
SQL
UPDATE employees
SET salary = 55000
WHERE emp_id = 1;
```

13. DELETE Data

```
SQL
DELETE FROM employees WHERE emp_id = 2;
```

14. Subquery (Advanced)

```
SQL
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```


15. Index (Performance)

```
SQL
CREATE INDEX idx_salary ON employees(salary);
```

16. View (Virtual Table)

```
SQL
CREATE VIEW high_salary AS
SELECT name, salary FROM employees WHERE salary > 50000;

SELECT * FROM high_salary;
```

17. Final Interview Answer

👉 "SQL allows us to create databases, define tables with constraints, insert and manipulate data, and perform advanced queries like joins, aggregations, and subqueries to extract meaningful insights."

Your Learning Path (Follow This)

1. Tables + Insert
2. SELECT + WHERE
3. ORDER BY + LIMIT
4. GROUP BY + HAVING
5. JOIN (very important )
6. Subqueries
7. Index + Views

If you want next level 

I can give you:

- ✅ 50 SQL practice questions (company-level)
- ✅ Real-world project (Instagram / Banking DB)
- ✅ Interview questions with answers (TCS, Infosys, Amazon)

You said:

🚀 Your Learning Path (Follow This) Tables + Insert SELECT + WHERE ORDER BY + LIMIT GROUP BY + HAVING JOIN (very important 🔥) Subqueries Index + Views If you want next level 🎯 I can give you: ✅ 50 SQL practice questions (company-level) explain all query

ChatGPT said:

Perfect — now we'll turn your roadmap into a **full learning guide with explanations + examples** so you *actually understand*, not just memorize 🚀

🚀 1. Tables + INSERT

📌 Create Table

```
SQL
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    salary INT,
    department VARCHAR(50)
);
```

📌 Insert Data

```
SQL
INSERT INTO employees VALUES
(1, 'Aamir', 50000, 'IT'),
(2, 'Rahul', 40000, 'HR'),
(3, 'Sara', 60000, 'IT');
```

🗨️ Explanation

- Table = structure (columns)

- INSERT = adds **rows (tuples)**

2. SELECT + WHERE

Get All Data

```
SQL
SELECT * FROM employees;
```

👉 * = all columns

Filter Data

```
SQL
SELECT * FROM employees WHERE salary > 50000;
```

Explanation

- WHERE filters data
- Only rows matching condition are returned

3. ORDER BY + LIMIT

Sorting

```
SQL
SELECT * FROM employees ORDER BY salary DESC;
```

👉 DESC = highest first

👉 ASC = lowest first

Limit Records

```
SQL
SELECT * FROM employees LIMIT 2;
```

Explanation

- LIMIT restricts number of rows

- Useful for **top results**

4. GROUP BY + HAVING

Group Data

```
SQL
SELECT department, COUNT(*)
FROM employees
GROUP BY department;
```

 Groups employees by department

Filter Groups

```
SQL
SELECT department, AVG(salary)
FROM employees
GROUP BY department
HAVING AVG(salary) > 45000;
```

Explanation

- GROUP BY = grouping
- HAVING = filter groups (like WHERE but after grouping)

5. JOIN (MOST IMPORTANT)

INNER JOIN

```
SQL
SELECT e.name, p.project_name
FROM employees e
INNER JOIN projects p
ON e.id = p.emp_id;
```

Explanation

- Combines data from two tables

- Only matching records

LEFT JOIN

```
SQL
SELECT e.name, p.project_name
FROM employees e
LEFT JOIN projects p
ON e.id = p.emp_id;
```

- 👉 Shows all employees (even without project)

6. Subqueries (Advanced)

Example

```
SQL
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Explanation

- Inner query runs first
- Outer query uses its result

7. Index (Performance)

```
SQL
CREATE INDEX idx_salary ON employees(salary);
```

Explanation

- Makes searching faster

- Like an index in a book

8. Views (Virtual Table)

```
SQL
CREATE VIEW high_salary AS
SELECT name, salary FROM employees WHERE salary > 50000;
```

```
SQL
SELECT * FROM high_salary;
```

Explanation

- View = saved query
- Acts like a table but stores no data

Final Understanding (Important)

Concept	Meaning
Table	Stores data
Row (Tuple)	One record
Column (Attribute)	Field
WHERE	Filter rows
GROUP BY	Group data
JOIN	Combine tables
Subquery	Query inside query
INDEX	Speed
VIEW	Virtual table

Interview Master Answer

 "SQL allows us to create tables, insert data, filter using WHERE, sort with ORDER BY, group using GROUP BY, combine tables using JOIN, perform advanced queries using subqueries, and optimize performance using indexes and

views.”

Next Step (Very Important)

If you really want to become strong:

I can give you:

👉 **50 real company-level SQL questions (TCS, Infosys, Amazon)**

👉 With **solutions + explanations step-by-step**

Just say: **“Give me 50 SQL questions”** 🚀

You said:

Give me 50 real company-level SQL questions

ChatGPT said:

Great — here are **50 real company-level SQL questions** (TCS, Infosys, Amazon, etc.) arranged from **basic** → **advanced** so you can practice step-by-step 🚀

BASIC LEVEL (1–15)

1.

Write a query to display all records from a table.

2.

Select only **name** and **salary** from employees.

3.

Find employees with salary greater than 50,000.

4.

Display employees from IT department.

5.

Find employees whose age is between 20 and 30.

6.

Show employees whose name starts with 'A'.

7.

Display unique departments.

8.

Find employees with NULL email.

9.

Sort employees by salary in descending order.

10.

Display top 5 highest-paid employees.

11.

Count total number of employees.

12.

Find maximum salary.

13.

Find minimum salary.

14.

Calculate average salary.

15.

Find total salary paid.



INTERMEDIATE LEVEL (16–30)

16.

Count employees in each department.

17.

Find average salary per department.

18.

Show departments with more than 5 employees.

19.

Find departments where average salary > 50,000.

20.

Display second highest salary.

21.

Find third highest salary.

22.

Get employees whose salary is above average.

23.

Find employees who do not belong to HR.

24.

Get employees whose name contains 'a'.

25.

Find employees hired in 2023.

26.

Display employee details with department names (JOIN).

27.

Find employees without any project.

28.

List all employees and their projects.

29.

Show departments with no employees.

30.

Find duplicate records in a table.

ADVANCED LEVEL (31–50)

31.

Find highest salary in each department.

32.

Find employee with highest salary in each department.

33.

Get top 3 highest-paid employees.

34.

Find employees earning more than their manager.

35.

Find employees who share same salary.

36.

Delete duplicate records.

37.

Find cumulative salary (running total).

38.

Find rank of employees based on salary.

39.

Find employees with odd/even ID.

40.

Get employees who joined last 30 days.

41.

Find difference between highest and lowest salary.

42.

Find employees with same name but different IDs.

43.

Pivot table (convert rows to columns).

44.

Unpivot data (columns to rows).

45.

Find missing IDs in sequence.

46.

Find employees who never logged in.

47.

Find most frequent salary.

48.

Get department with highest average salary.

49.

Find employees working on multiple projects.

50.

Write a query using CTE (Common Table Expression).



How to Practice (Important)

👉 Don't try all at once

Follow this:

- Day 1 → Q1–10
- Day 2 → Q11–20
- Day 3 → Q21–30
- Day 4 → Q31–40
- Day 5 → Q41–50



Next Step

If you want real improvement, I suggest:

👉 I can give you

✅ **All 50 solutions with explanation**

✅ OR solve them **one by one like interview practice**

✅ OR give **mini project (Instagram DB / Banking DB)**

Just tell me:

👉 "Give me solutions" or

👉 "Start solving step-by-step"

You said:

Give me 50 real company-level SQL questions with solution

ChatGPT said:

Good — this is exactly how you level up 🚀

I'll give you **all 50 SQL questions with solutions + short explanations** (clean + interview-ready).

🟢 BASIC LEVEL (1–15)

1. All records

```
SQL
SELECT * FROM employees;
```

👉 Fetches all columns + rows

2. Specific columns

```
SQL
SELECT name, salary FROM employees;
```

3. Salary > 50000

```
SQL
SELECT * FROM employees WHERE salary > 50000;
```

4. IT department

```
SQL
SELECT * FROM employees WHERE department = 'IT';
```

5. Age between 20–30

```
SQL
SELECT * FROM employees WHERE age BETWEEN 20 AND 30;
```

6. Name starts with A

```
SQL
SELECT * FROM employees WHERE name LIKE 'A%';
```

7. Unique departments

```
SQL
SELECT DISTINCT department FROM employees;
```

8. NULL email

```
SQL
SELECT * FROM employees WHERE email IS NULL;
```

9. Sort by salary

```
SQL
SELECT * FROM employees ORDER BY salary DESC;
```

10. Top 5 salary

```
SQL
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

11. Count employees

```
SQL
SELECT COUNT(*) FROM employees;
```

12. Max salary

```
SQL
SELECT MAX(salary) FROM employees;
```

13. Min salary

```
SQL
SELECT MIN(salary) FROM employees;
```

14. Avg salary

```
SQL
SELECT AVG(salary) FROM employees;
```

15. Total salary

```
SQL
SELECT SUM(salary) FROM employees;
```

INTERMEDIATE (16–30)

16. Count per department

```
SQL
SELECT department, COUNT(*)
FROM employees
GROUP BY department;
```

17. Avg salary per department

```
SQL
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

18. Dept > 5 employees

```
SQL
SELECT department, COUNT(*)
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

19. Avg salary > 50000

```
SQL
SELECT department, AVG(salary)
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;
```

20. Second highest salary

```
SQL
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

21. Third highest salary

```
SQL
SELECT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

22. Salary above average

```
SQL
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

23. Not HR

```
SQL
SELECT * FROM employees WHERE department != 'HR';
```

24. Name contains 'a'

```
SQL
SELECT * FROM employees WHERE name LIKE '%a%';
```

25. Hired in 2023

```
SQL
SELECT * FROM employees
WHERE YEAR(join_date) = 2023;
```

26. JOIN employee + department

```
SQL
SELECT e.name, d.dept_name
FROM employees e
JOIN departments d
ON e.department = d.dept_name;
```

27. Employees without project

```
SQL
SELECT * FROM employees e
LEFT JOIN projects p ON e.id = p.emp_id
WHERE p.emp_id IS NULL;
```


28. Employees + projects

```
SQL
SELECT e.name, p.project_name
FROM employees e
LEFT JOIN projects p ON e.id = p.emp_id;
```

29. Departments without employees

```
SQL
SELECT d.dept_name
FROM departments d
LEFT JOIN employees e
ON d.dept_name = e.department
WHERE e.id IS NULL;
```

30. Duplicate records

```
SQL
SELECT name, COUNT(*)
FROM employees
GROUP BY name
HAVING COUNT(*) > 1;
```

ADVANCED (31–50)

31. Highest salary per dept

```
SQL
SELECT department, MAX(salary)
FROM employees
GROUP BY department;
```

32. Highest salary employee per dept

```
SQL
SELECT *
FROM employees e
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
    WHERE department = e.department
);
```

33. Top 3 employees

```
SQL
SELECT * FROM employees
ORDER BY salary DESC
LIMIT 3;
```

34. Salary > manager

```
SQL
SELECT e.name
FROM employees e
JOIN employees m
ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

35. Same salary employees

```
SQL
SELECT salary, COUNT(*)
FROM employees
GROUP BY salary
HAVING COUNT(*) > 1;
```

36. Delete duplicates

```
SQL
DELETE FROM employees
WHERE id NOT IN (
    SELECT MIN(id)
    FROM employees
    GROUP BY name
);
```

37. Running total

```
SQL
SELECT name, salary,
SUM(salary) OVER (ORDER BY id) AS running_total
FROM employees;
```

38. Rank employees

```
SQL
SELECT name, salary,
RANK() OVER (ORDER BY salary DESC) AS rank
FROM employees;
```

39. Odd/Even ID

```
SQL
SELECT * FROM employees WHERE id % 2 = 0; -- even
SELECT * FROM employees WHERE id % 2 = 1; -- odd
```

40. Last 30 days

```
SQL
SELECT * FROM employees
WHERE join_date >= CURDATE() - INTERVAL 30 DAY;
```

41. Salary difference

```
SQL
SELECT MAX(salary) - MIN(salary) FROM employees;
```

42. Same name diff ID

```
SQL
SELECT name
FROM employees
GROUP BY name
HAVING COUNT(DISTINCT id) > 1;
```

43. Pivot (Example)

```
SQL
SELECT
SUM(CASE WHEN department='IT' THEN salary END) AS IT,
SUM(CASE WHEN department='HR' THEN salary END) AS HR
FROM employees;
```

44. Unpivot

```
SQL
SELECT id, 'salary' AS field, salary FROM employees
UNION ALL
SELECT id, 'age', age FROM employees;
```

45. Missing IDs

```
SQL
SELECT t1.id + 1
FROM employees t1
LEFT JOIN employees t2
ON t1.id + 1 = t2.id
WHERE t2.id IS NULL;
```

46. Never logged in

```
SQL
SELECT * FROM users
WHERE last_login IS NULL;
```

47. Most frequent salary

```
SQL
SELECT salary
FROM employees
GROUP BY salary
ORDER BY COUNT(*) DESC
LIMIT 1;
```

48. Dept highest avg salary

```
SQL
SELECT department
FROM employees
GROUP BY department
ORDER BY AVG(salary) DESC
LIMIT 1;
```

49. Multiple projects

```
SQL
SELECT emp_id
FROM projects
GROUP BY emp_id
HAVING COUNT(*) > 1;
```

50. CTE

```
SQL
WITH avg_sal AS (
    SELECT AVG(salary) AS avg_salary FROM employees
)
SELECT * FROM employees
WHERE salary > (SELECT avg_salary FROM avg_sal);
```



Final Advice (Important)

If you really want placement:

👉 Focus on:

- JOIN (🔥 must)

- GROUP BY + HAVING
- Subqueries
- Window functions

Next Step

I can take you to **next level (real interview mode)**:

- 👉 Solve these **live with you step-by-step**
- 👉 OR give you **Mini Project (Instagram / Banking DB)**
- 👉 OR give **Top 20 Most Asked Questions only**

Just say:

- 👉 *"Start interview practice"* or
- 👉 *"Give project"* 🚀

You said:

Give me the Top 20 Most Asked Questions only

ChatGPT said:

Here are the 🔥 **Top 20 Most Asked SQL Interview Questions** (very important for TCS, Infosys, Amazon, etc.) with **clean solutions + quick explanations**.

TOP 20 SQL INTERVIEW QUESTIONS

1. Find Second Highest Salary

```
SQL
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

- 👉 Excludes highest, then finds next max

2. Find Third Highest Salary

```
SQL
SELECT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

👉 Skip first 2, get 3rd

3. Get Employees Above Average Salary

```
SQL
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

4. Count Employees per Department

```
SQL
SELECT department, COUNT(*)
FROM employees
GROUP BY department;
```

5. Departments with More Than 5 Employees

```
SQL
SELECT department, COUNT(*)
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

6. Highest Salary in Each Department

```
SQL
SELECT department, MAX(salary)
FROM employees
GROUP BY department;
```

7. Employee with Highest Salary per Department

```
SQL
SELECT *
FROM employees e
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
    WHERE department = e.department
);
```

8. Find Duplicate Records

```
SQL
SELECT name, COUNT(*)
FROM employees
GROUP BY name
HAVING COUNT(*) > 1;
```

9. Delete Duplicate Records

```
SQL
DELETE FROM employees
WHERE id NOT IN (
    SELECT MIN(id)
    FROM employees
    GROUP BY name
);
```

10. Employees Without Projects

```
SQL
SELECT * FROM employees e
LEFT JOIN projects p ON e.id = p.emp_id
WHERE p.emp_id IS NULL;
```


11. INNER JOIN Example

```
SQL
SELECT e.name, p.project_name
FROM employees e
JOIN projects p
ON e.id = p.emp_id;
```

12. LEFT JOIN Example

```
SQL
SELECT e.name, p.project_name
FROM employees e
LEFT JOIN projects p
ON e.id = p.emp_id;
```

13. Top 3 Highest Paid Employees

```
SQL
SELECT * FROM employees
ORDER BY salary DESC
LIMIT 3;
```

14. Employees Earning More Than Manager

```
SQL
SELECT e.name
FROM employees e
JOIN employees m
ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

15. Rank Employees by Salary

```
SQL
SELECT name, salary,
RANK() OVER (ORDER BY salary DESC) AS rank
FROM employees;
```

16. Running Total (Cumulative Salary)

```
SQL
SELECT name, salary,
SUM(salary) OVER (ORDER BY id) AS running_total
FROM employees;
```

17. Most Frequent Salary

```
SQL
SELECT salary
FROM employees
GROUP BY salary
ORDER BY COUNT(*) DESC
LIMIT 1;
```

18. Department with Highest Avg Salary

```
SQL
SELECT department
FROM employees
GROUP BY department
ORDER BY AVG(salary) DESC
LIMIT 1;
```

19. Employees Working on Multiple Projects

```
SQL
SELECT emp_id
FROM projects
GROUP BY emp_id
HAVING COUNT(*) > 1;
```

20. CTE (Common Table Expression)

```
SQL
WITH avg_sal AS (
    SELECT AVG(salary) AS avg_salary FROM employees
)
SELECT * FROM employees
WHERE salary > (SELECT avg_salary FROM avg_sal);
```

Final Strategy (Very Important)

👉 If you prepare only these 20:

- 70–80% interview coverage ✅
- Focus on:
 - JOIN 🔥
 - GROUP BY + HAVING
 - Subqueries
 - Window functions

Next Level

If you're serious about cracking interviews:

I can:

- ✅ Take **mock SQL interview (like real company)**
- ✅ Give **1 real project (Instagram DB / Banking DB)**
- ✅ Teach **advanced JOIN + window functions deeply**

Just say:

- 👉 **"Start mock interview"** or
- 👉 **"Give project"** 💯

You said:

Table Queries • Create • Insert • Update • Alter • Truncate • Delete explain all

ChatGPT said:

You're asking about the **core table queries in SQL** — these are super important for interviews and real work. Let's break each one in a **simple + clear way** 👉

1. CREATE (Create Table)

👉 Used to **create a new table**

```
SQL
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT
);
```

Explanation

- Defines structure (columns + data types)
- Table is empty initially

2. INSERT (Add Data)

 Used to **add data into table**

```
SQL
INSERT INTO students VALUES (1, 'Aamir', 22);
```

```
SQL
INSERT INTO students (id, name, age)
VALUES (2, 'Rahul', 21);
```

Explanation

- Adds **rows (tuples)**
- Can insert single or multiple records

3. UPDATE (Modify Data)

 Used to **change existing data**

```
SQL
UPDATE students
SET age = 23
WHERE id = 1;
```

Explanation

- **WHERE** is important ⚠️

- Without WHERE → all rows updated

4. ALTER (Change Table Structure)

 Used to **modify table structure**

Add Column

```
SQL
ALTER TABLE students ADD email VARCHAR(50);
```

Drop Column

```
SQL
ALTER TABLE students DROP COLUMN email;
```

Modify Column

```
SQL
ALTER TABLE students MODIFY age INT;
```

Explanation

- Changes table design (not data directly)

5. TRUNCATE (Delete All Data Fast)

 Used to **remove all data from table**

```
SQL
TRUNCATE TABLE students;
```

Explanation

- Deletes all rows quickly 
- Cannot use WHERE
- Resets auto-increment

- Faster than DELETE

6. DELETE (Remove Specific Data)

 Used to **delete rows**

```
SQL
DELETE FROM students WHERE id = 1;
```

```
SQL
DELETE FROM students;
```

Explanation

- With WHERE → deletes specific rows
- Without WHERE → deletes all rows
- Slower than TRUNCATE

IMPORTANT DIFFERENCE (Interview)

Feature	DELETE	TRUNCATE
WHERE clause	 Yes	 No
Speed	Slow	Fast
Rollback	Possible	Not usually
Deletes	Selected rows	All rows

Final Interview Answer

 "SQL table queries include CREATE to define tables, INSERT to add data, UPDATE to modify data, ALTER to change structure, DELETE to remove specific rows, and TRUNCATE to quickly remove all data from a table."

Next Step

If you want strong command in SQL:

I can give you:

-  **Real practice questions on each query**
-  **Mini project (student system / Instagram DB)**

✅ Mistakes to avoid in interviews

Just say: **"Give practice on these queries"** ¹⁰⁰

You said:

Constraints lles for data in the table NOT NULL columns cannot have a null value UNIQUE all values in column are different DEFAULT sets the default value of a column CHECK it can limit the values allowed in a column salary INT DEFAULT 25000 CONSTRAINT age_check CHECK (age >= 18 AND city="Delhi")

ChatGPT said:

You're very close 👍 — let me clean this up and explain **SQL Constraints properly (simple + interview-ready)**.

📌 What are Constraints?

👉 **Constraints are rules applied on table columns to control the type of data that can be stored.**

👉 In simple words:
They ensure data is correct, valid, and safe.

📌 Types of Constraints

1. NOT NULL

👉 Column **cannot store NULL (empty) values**

```
SQL
name VARCHAR(50) NOT NULL
```

💬 Example

- ❌ name = NULL → not allowed
- ✅ name = "Aamir" → allowed

2. UNIQUE

👉 All values in the column must be **different**

```
SQL
email VARCHAR(50) UNIQUE
```

Example

-  Same email twice → not allowed
-  All emails different → allowed

3. DEFAULT

 Sets a **default value** if no value is provided

```
SQL
salary INT DEFAULT 25000
```

Example

- If salary not given → automatically **25000**

4. CHECK

 Restricts values based on a condition

```
SQL
CHECK (age >= 18)
```

Example

-  age = 16 → rejected
-  age = 20 → allowed

Your Example (Corrected)

```
SQL
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT,
    city VARCHAR(50),
    salary INT DEFAULT 25000,
    CONSTRAINT age_check CHECK (age >= 18 AND city = 'Delhi')
);
```


Explanation of Your CHECK Constraint

```
SQL
CHECK (age >= 18 AND city = 'Delhi')
```

👉 Means:

- Age must be **18 or more**
- City must be **Delhi**

Important Notes (Interview 🔥)

- Constraints improve **data integrity**
- Can be applied at:
 - Column level
 - Table level
- Common constraints:
 - NOT NULL
 - UNIQUE
 - PRIMARY KEY 🔥
 - FOREIGN KEY 🔥
 - CHECK
 - DEFAULT

Interview Answer

👉 *"Constraints are rules applied on table columns to ensure data accuracy and integrity. Examples include NOT NULL, UNIQUE, DEFAULT, and CHECK."*

Pro Tip (Important)

👉 Your CHECK condition:

```
SQL
age >= 18 AND city = 'Delhi'
```

⚠️ This forces **only Delhi users allowed** — not always practical

👉 Better version:

```
SQL
CHECK (age >= 18)
```

🚀 Next Step

If you want mastery:

I can teach you:

- ✅ PRIMARY KEY vs FOREIGN KEY (most asked 🔥)
- ✅ Real constraint-based interview questions
- ✅ Mistakes students make in SQL

Just say: **“Explain keys”** 💯

You said:

Constraints • PRIMARY KEY makes a column unique & not null but used only for one CREATE TABLE temp (id int not null, PRIMARY KEY (id)); FOREIGN KEY prevent actions that would destroy links between tables CREATE TABLE temp (cust_id int, FOREIGN KEY (cust_id) references customer (id));

ChatGPT said:

You're on the right track 👍 — let's clean this up and explain **PRIMARY KEY & FOREIGN KEY** clearly (simple + interview-ready).

📌 1. PRIMARY KEY

👉 A **PRIMARY KEY** is a column (or combination of columns) that:

- **Uniquely identifies each row**

- **Cannot be NULL**

✓ Correct Syntax

```
SQL
CREATE TABLE temp (
  id INT NOT NULL,
  PRIMARY KEY (id)
);
```

🗨 Key Points

- No duplicate values ❌
- No NULL values ❌
- Only **one primary key per table** (but can be composite)

📌 Example

id name

1 Aamir

2 Rahul

✓ id is unique → valid

❌ duplicate id → not allowed

📌 2. FOREIGN KEY

👉 A **FOREIGN KEY** is used to:

- **Link two tables**
- Maintain **relationship between them**

👉 It ensures that values must exist in the parent table

✓ Correct Syntax

```
SQL
CREATE TABLE customer (
    id INT PRIMARY KEY,
    name VARCHAR(50)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    cust_id INT,
    FOREIGN KEY (cust_id) REFERENCES customer(id)
);
```

💡 Key Points

- `cust_id` must exist in `customer.id`
- Prevents invalid data (no fake IDs)

📌 Example

customer table

id	name
----	------

1	Aamir
---	-------

2	Rahul
---	-------

orders table

order_id	cust_id
----------	---------

101	1
-----	---

102	5
-----	---

📌 Why FOREIGN KEY is Important?

👉 Prevents:

- Deleting a parent row that is still referenced

- Inserting invalid references

⚡ PRIMARY KEY vs FOREIGN KEY (Interview 🔥)

Feature	PRIMARY KEY	FOREIGN KEY
Purpose	Unique ID	Link tables
NULL allowed	❌ No	✅ Yes (sometimes)
Duplicates	❌ No	✅ Allowed
Count per table	One	Multiple

🎯 Final Interview Answer

👉 "A primary key uniquely identifies each record in a table and cannot be null, while a foreign key is used to create relationships between tables by referencing the primary key of another table."

🚀 Next Step (Very Important)

You should now learn:

- 👉 Joins (because they use foreign keys) 🔥
- 👉 Normalization (real DB design)
- 👉 Real-world schema design (Instagram / Banking)

Just say: "Explain joins deeply" 💯

You said:

Final Interview Answer 👉 "A primary key uniquely identifies each record in a table and cannot be null, while a foreign key is used to create relationships between tables by referencing the primary key of another table." 🚀 Next Step (Very Important) You should now learn: 👉 Joins (because they use foreign keys) 🔥 👉 Normalization (real DB design) through example easy way to understand

ChatGPT said:

Good—this is exactly the right next step. I'll explain **JOINS + Normalization** in a **simple, practical way with examples** so it actually sticks.

PART 1: JOINS (Very Important)

 **JOIN = combine data from multiple tables**

Think:

You have data split across tables → JOIN helps you **connect and fetch it together**

Example Tables

customers

id	name
----	------

1	Aamir
---	-------

2	Rahul
---	-------

3	Sara
---	------

orders

order_id	cust_id	product
----------	---------	---------

101	1	Laptop
-----	---	--------

102	2	Phone
-----	---	-------

103	1	Mouse
-----	---	-------

INNER JOIN

 Shows **only matching data**

```
SQL
SELECT c.name, o.product
FROM customers c
INNER JOIN orders o
ON c.id = o.cust_id;
```

Result

name	product
------	---------

Aamir	Laptop
-------	--------

name	product
Rahul	Phone
Aamir	Mouse

✓ Only customers who placed orders

2 LEFT JOIN

👉 Shows **all from left table + matching from right**

```
SQL
SELECT c.name, o.product
FROM customers c
LEFT JOIN orders o
ON c.id = o.cust_id;
```

Result

name	product
Aamir	Laptop
Rahul	Phone
Sara	NULL

✓ Sara has no order → still shown

3 RIGHT JOIN

👉 Opposite of LEFT JOIN

```
SQL
SELECT c.name, o.product
FROM customers c
RIGHT JOIN orders o
ON c.id = o.cust_id;
```

✓ Shows all orders even if no customer

4 FULL JOIN (Concept)

👉 Shows **all data from both tables**

(MySQL doesn't support directly — done via UNION)

JOIN Interview Line

👉 "JOIN is used to combine rows from two or more tables based on a related column like a foreign key."

PART 2: NORMALIZATION (Easy Way)

👉 **Normalization = organizing data to remove redundancy (duplicate data)**

👉 Goal:

- No duplicate data ❌
- Better structure ✅
- Easy updates ✅

❌ Without Normalization (Bad Design)

student_id	name	course	instructor
1	Aamir	Java	Ravi
2	Rahul	Java	Ravi
3	Sara	Python	Neha

Problems:

- "Ravi" repeated many times
- If instructor changes → update everywhere ❌

✅ With Normalization (Good Design)

students

id	name
1	Aamir
2	Rahul
3	Sara

courses

id	course	instructor
----	--------	------------

101	Java	Ravi
-----	------	------

102	Python	Neha
-----	--------	------

student_course

student_id	course_id
------------	-----------

1	101
---	-----

2	101
---	-----

3	102
---	-----

Benefits

- ✓ No repetition
- ✓ Easy updates
- ✓ Data consistency

Types of Normalization (Interview)

- **1NF** → No repeating groups
- **2NF** → Remove partial dependency
- **3NF** → Remove transitive dependency

👉 (For freshers: just explain basic idea 👍)

Final Interview Answer

👉 "Joins are used to combine data from multiple tables using common columns like foreign keys. Normalization is the process of organizing data into multiple tables to reduce redundancy and improve data integrity."

What You Should Do Next

To become strong:

👉 Practice:

- INNER JOIN 🔥

- LEFT JOIN 🔥
- GROUP BY + JOIN

100 Next Level

I can now:

- ✓ Give **10 real JOIN interview questions with solutions**
- ✓ OR build **Instagram database project step-by-step**
- ✓ OR take **mock SQL interview**

Just say:

- 👉 **"Give JOIN questions"** or
- 👉 **"Start project"** 🚀

You said:

What are Keys? Keys are special columns in the table Primary Key It is a column (or set of columns) in a table that uniquely identifies each row. (a unique id) There is only 1 PK & it should be NOT null. Foreign Key A foreign key is a column (or set of columns) in a table that refers to the primary key in FKs can have duplicate & null values. There can be multiple FKs.

ChatGPT said:

You've got the idea 👍 — let me refine and explain **Keys in SQL** in a clean, correct, interview-ready way.

📌 What are Keys?

👉 **Keys are special columns (or set of columns) used to uniquely identify rows and create relationships between tables.**

👉 In simple words:

Keys help identify data and connect tables.

🔑 Types of Keys

1 Primary Key (PK)

👉 A **Primary Key**:

- Uniquely identifies each row

- Cannot be NULL ❌
- Cannot have duplicate values ❌
- Only **one primary key per table**

✅ Example

```
SQL
CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(50)
);
```

💬 Example Data

id name

1 Aamir

2 Rahul

✓ Unique → valid

❌ Duplicate or NULL → not allowed

2 Foreign Key (FK)

👉 A **Foreign Key**:

- Links one table to another
- Refers to **Primary Key of another table**
- Can have **duplicates** ✅

- Can have **NULL values** 

 Example

```
SQL
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    student_id INT,
    FOREIGN KEY (student_id) REFERENCES students(id)
);
```

 Example Data

students

id	name
1	Aamir
2	Rahul

orders

order_id	student_id
101	1
102	1
103	NULL
104	5

 Key Differences (Important)

Feature	Primary Key	Foreign Key
Purpose	Unique ID	Link tables
NULL allowed	 No	 Yes

Feature	Primary Key	Foreign Key
Duplicate allowed	✗ No	✓ Yes
Count per table	One	Many

Other Important Keys (Short)

👉 Just for knowledge (sometimes asked):

- **Candidate Key** → Possible primary keys
- **Super Key** → Any unique combination
- **Composite Key** → Multiple columns as key

Final Interview Answer

👉 *"Keys are special columns used to uniquely identify records and establish relationships between tables. A primary key uniquely identifies each row and cannot be null, while a foreign key references a primary key in another table and is used to maintain relationships."*

Next Step

Now you should master:

- 👉 **JOINS using keys (most important 🔥)**
- 👉 **Real schema design (projects)**
- 👉 **Advanced queries using relationships**

Just say:

- 👉 **"Give JOIN practice questions"** or
- 👉 **"Start real database project" 💯**

You said:

our first database

ChatGPT said:

Great—let's build **your first database from scratch** step by step (simple but real-world) 

Step 1: Create Database

```
SQL
CREATE DATABASE first_db;
USE first_db;
```

👉 Creates and selects your database

Step 2: Create Tables

Users Table

```
SQL
CREATE TABLE users (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  age INT CHECK (age >= 18)
);
```

Orders Table (with Foreign Key)

```
SQL
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  product VARCHAR(50),
  user_id INT,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

👉 This creates a **relationship** between users and orders

Step 3: Insert Data

```
SQL
INSERT INTO users VALUES
(1, 'Aamir', 'aamir@gmail.com', 22),
(2, 'Rahul', 'rahul@gmail.com', 21),
(3, 'Sara', 'sara@gmail.com', 23);
```

```
SQL
INSERT INTO orders VALUES
(101, 'Laptop', 1),
(102, 'Phone', 2),
(103, 'Mouse', 1);
```

Step 4: View Data

```
SQL
SELECT * FROM users;
SELECT * FROM orders;
```

Step 5: JOIN (Combine Data)

```
SQL
SELECT u.name, o.product
FROM users u
JOIN orders o
ON u.id = o.user_id;
```

Output

name	product
------	---------

Aamir	Laptop
-------	--------

Rahul	Phone
-------	-------

Aamir	Mouse
-------	-------

Step 6: Filter Data

```
SQL
SELECT * FROM users WHERE age > 21;
```

Step 7: Update Data

```
SQL
UPDATE users
SET age = 24
WHERE id = 1;
```

Step 8: Delete Data

```
SQL
DELETE FROM users WHERE id = 3;
```

What You Learned

- ✓ Create database
- ✓ Create tables
- ✓ Primary key & foreign key
- ✓ Insert data
- ✓ SELECT & WHERE
- ✓ JOIN (very important 🔥)
- ✓ UPDATE & DELETE

Final Interview Line

👉 *"In my first database project, I created tables with primary and foreign keys, inserted data, and used SQL queries like SELECT, JOIN, UPDATE, and DELETE to manage and retrieve data."*

Next Step (Important)

Now you're ready for:

- 👉 **Mini Project (Instagram Database 🔥)**
- 👉 **Banking System Database**
- 👉 **Advanced JOIN + Queries**

Just say:

- 👉 **"Start Instagram project"** or
- 👉 **"Give next level project"** 🚀

You said:

our first table

ChatGPT said:

Nice—now let's build **your first table** step by step (super simple + clear) 🚀

Step 1: Create Database

```
SQL
CREATE DATABASE first_db;
USE first_db;
```

Step 2: Create Your First Table

```
SQL
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT
);
```

Explanation

- **id** → unique identifier (**Primary Key**)
- **name** → cannot be empty (**NOT NULL**)
- **age** → normal column

👉 This table will store **student data**

Step 3: Insert Data

```
SQL
INSERT INTO students VALUES
(1, 'Aamir', 22),
(2, 'Rahul', 21),
(3, 'Sara', 23);
```

Step 4: View Table Data

```
SQL
SELECT * FROM students;
```

Output

id	name	age
1	Aamir	22
2	Rahul	21
3	Sara	23

Step 5: Simple Queries

Filter Data

```
SQL
SELECT * FROM students WHERE age > 21;
```

Select Specific Columns

```
SQL
SELECT name, age FROM students;
```

What You Learned

- ✓ Create table
- ✓ Add data (INSERT)
- ✓ View data (SELECT)
- ✓ Filter data (WHERE)

Interview Line

 "A table is a collection of data organized in rows and columns. I created my first table using *CREATE TABLE*, inserted data using *INSERT*, and retrieved data using *SELECT*."

Next Step

Now you should learn:

- 👉 **Constraints (NOT NULL, UNIQUE, DEFAULT)**
- 👉 **Keys (Primary + Foreign)**
- 👉 **Joins (very important 🔥)**

Just say:

- 👉 **"Next level table" or**
- 👉 **"Give practice questions on table" *100***

You said:

database query

ChatGPT said:

What is a Database Query?

👉 A **database query** is a command used to **interact with a database**.

👉 In simple words:

Query = asking the database to do something (get, add, update, delete data)

Types of Database Queries

1 **SELECT (Read Data)**

👉 Used to **fetch data**

```
SQL
SELECT * FROM students;
```

✓ Gets all data

```
SQL
SELECT name, age FROM students;
```

✓ Gets specific columns

2 INSERT (Add Data)

👉 Used to **insert new records**

```
SQL
INSERT INTO students VALUES (4, 'Ali', 20);
```

3 UPDATE (Modify Data)

👉 Used to **change existing data**

```
SQL
UPDATE students
SET age = 21
WHERE id = 4;
```

4 DELETE (Remove Data)

👉 Used to **delete records**

```
SQL
DELETE FROM students WHERE id = 4;
```

5 CREATE (Create Table/Database)

```
SQL
CREATE TABLE students (
    id INT,
    name VARCHAR(50)
);
```

6 DROP (Delete Table/Database)

```
SQL
DROP TABLE students;
```

7 ALTER (Modify Table)

```
SQL
ALTER TABLE students ADD email VARCHAR(50);
```

Important Query Clauses

WHERE (Filter)

```
SQL
SELECT * FROM students WHERE age > 20;
```

ORDER BY (Sort)

```
SQL
SELECT * FROM students ORDER BY age DESC;
```

GROUP BY (Group Data)

```
SQL
SELECT age, COUNT(*) FROM students GROUP BY age;
```

LIMIT (Restrict Results)

```
SQL
SELECT * FROM students LIMIT 2;
```

Final Interview Answer

 "A database query is a command used to interact with a database to retrieve, insert, update, or delete data. Common queries include *SELECT*, *INSERT*, *UPDATE*, and *DELETE*."

Next Step

To become strong in SQL:

 Practice:

- SELECT + WHERE 🔥
- GROUP BY + HAVING

- JOIN (most important)

If you want:

- ✓ Practice questions
- ✓ Real interview queries
- ✓ Mini project

Just say: **"Give query practice"** ¹⁰⁰

You said:

create table

ChatGPT said:

CREATE TABLE in SQL

 **CREATE TABLE** is used to **create a new table** in a database.

 In simple words:

It defines structure (columns + data types) where data will be stored.

Basic Syntax

```
SQL
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype
);
```

Example (Your First Table)

```
SQL
CREATE TABLE students (
    id INT,
    name VARCHAR(50),
    age INT
);
```

Explanation

- **id** → number
- **name** → text (max 50 characters)
- **age** → number

With Constraints (Important)

```
SQL
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INT CHECK (age >= 18),
    salary INT DEFAULT 25000
);
```

What's happening:

- **PRIMARY KEY** → unique + not null
- **NOT NULL** → cannot be empty
- **UNIQUE** → no duplicate values
- **CHECK** → condition applied
- **DEFAULT** → auto value if not given

Create Table with Foreign Key

```
SQL
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    product VARCHAR(50),
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES students(id)
);
```

 Connects two tables (relationship)

Check Table

```
SQL  
SHOW TABLES;
```

```
SQL  
DESCRIBE students;
```

Interview Answer

👉 *"CREATE TABLE is used to define a new table in a database by specifying column names, data types, and constraints like primary key and foreign key."*

Next Step

Now you should learn:

- 👉 INSERT (add data)
- 👉 SELECT (get data)
- 👉 WHERE + JOIN 🔥

If you want:

- ✅ Practice questions on CREATE TABLE
- ✅ Real project (Instagram DB)
- ✅ Advanced schema design

Just say: **"Next step INSERT"** 💯

You said:

what are constraints

ChatGPT said:

What are Constraints in SQL?

👉 **Constraints are rules applied to table columns to control what data can be stored.**

👉 In simple words:

They ensure the data is correct, valid, and consistent.

Why Constraints are Important?

- Prevent wrong data ❌
- Maintain data accuracy ✅
- Improve data integrity 🔥

Types of Constraints

1 NOT NULL

👉 Column **cannot have empty (NULL) values**

```
SQL
name VARCHAR(50) NOT NULL
```

2 UNIQUE

👉 All values must be **different**

```
SQL
email VARCHAR(50) UNIQUE
```

3 PRIMARY KEY 🔥

👉 Combination of:

- UNIQUE ✅
- NOT NULL ✅

```
SQL
id INT PRIMARY KEY
```

4 FOREIGN KEY 🔥

👉 Used to **connect two tables**

```
SQL
FOREIGN KEY (user_id) REFERENCES users(id)
```

5 DEFAULT

👉 Sets a **default value**

```
SQL
salary INT DEFAULT 25000
```

6 CHECK

👉 Restricts values using a condition

```
SQL
CHECK (age >= 18)
```

📌 Full Example

```
SQL
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INT CHECK (age >= 18),
    salary INT DEFAULT 25000
);
```

🎯 Final Interview Answer

👉 *"Constraints are rules applied to table columns to ensure data accuracy and integrity. Examples include NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, and DEFAULT."*

🚀 Next Step

Now you should practice:

- 👉 Writing tables with constraints 🔥
- 👉 Using PRIMARY KEY & FOREIGN KEY
- 👉 JOIN queries

If you want:

- ✅ Practice questions
- ✅ Mini project
- ✅ Interview questions

Just say: **"Give constraint practice"** ¹⁰⁰

You said:

key constraints

ChatGPT said:

Key Constraints in SQL

👉 **Key constraints** are rules used to **identify records uniquely and maintain relationships between tables**.

👉 In simple words:

They help keep data unique and connected correctly.

Main Key Constraints

1 PRIMARY KEY 🔥

👉 A **Primary Key**:

- Uniquely identifies each row
- Cannot be NULL ❌
- Cannot have duplicates ❌
- Only **one per table**

✅ **Example**

```
SQL
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50)
);
```

Meaning

- Each student has a **unique id**

- No two rows can have same id

2 FOREIGN KEY 🔥

👉 A Foreign Key:

- Connects two tables
- Refers to **Primary Key of another table**
- Can have duplicates ✅
- Can have NULL values ✅

✅ Example

```
SQL
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    student_id INT,
    FOREIGN KEY (student_id) REFERENCES students(id)
);
```

💡 Meaning

- `student_id` must exist in `students.id`
- Maintains relationship

📌 Other Important Keys (Short)

3 UNIQUE KEY

- 👉 Ensures all values are different

```
SQL
email VARCHAR(100) UNIQUE
```

4 COMPOSITE KEY

- 👉 Combination of multiple columns as key

```
SQL
PRIMARY KEY (id, email)
```

5 CANDIDATE KEY

👉 Possible columns that can be primary key

📌 Comparison (Important 🔥)

Feature	Primary Key	Foreign Key
Purpose	Unique ID	Link tables
NULL allowed	❌ No	✅ Yes
Duplicate allowed	❌ No	✅ Yes
Count	One	Many

🎯 Interview Answer

👉 *"Key constraints are used to uniquely identify records and maintain relationships between tables. The primary key uniquely identifies each row, while the foreign key links tables by referencing a primary key."*

🚀 Next Step

Now you should learn:

- 👉 **JOIN using keys (most important 🔥)**
- 👉 **Real database design**
- 👉 **Advanced queries**

If you want:

- ✅ JOIN practice questions
- ✅ Real project (Instagram DB)
- ✅ Mock interview

Just say: **"Give JOIN practice"** 💯

You said:

primary and foreign keys

ChatGPT said:

Primary Key vs Foreign Key (Simple + Clear)

1. Primary Key (PK)

👉 A **Primary Key** is a column (or columns) that:

- **Uniquely identifies each row**
- **Cannot be NULL**
- **Cannot have duplicate values**
- Only **one primary key per table**

✅ Example

```
SQL
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50)
);
```

Example Data

id	name
----	------

1	Aamir
---	-------

2	Rahul
---	-------

✓ Each **id** is unique

✗ Duplicate or NULL → not allowed

2. Foreign Key (FK)

👉 A **Foreign Key** is a column that:

- **Creates a link between two tables**
- Refers to **Primary Key of another table**
- Can have **duplicate values**

- Can have **NULL values**

✓ Example

```
SQL
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    student_id INT,
    FOREIGN KEY (student_id) REFERENCES students(id)
);
```

💬 Example Data

students

id	name
----	------

1	Aamir
---	-------

2	Rahul
---	-------

orders

order_id	student_id
----------	------------

101	1
-----	---

102	1
-----	---

103	NULL
-----	------

104	5
-----	---

⚡ Key Differences (Interview 🔥)

Feature	Primary Key	Foreign Key
Purpose	Unique ID	Link tables
NULL allowed	✗ No	✓ Yes

Feature	Primary Key	Foreign Key
Duplicate allowed	✗ No	✓ Yes
Count per table	One	Multiple

Final Interview Answer

👉 "A primary key uniquely identifies each record in a table and cannot be null or duplicate. A foreign key is used to create relationships between tables by referencing the primary key of another table."

Next Step

To fully understand keys, you should now learn:

- 👉 **JOIN using primary & foreign key (🔥 most important)**
- 👉 **Real database design (projects)**

If you want:

- ✓ 10 JOIN practice questions
- ✓ Mini project (Instagram DB)
- ✓ Mock interview

Just say: **"Give JOIN questions"** 💯

You said:

insert into table

ChatGPT said:

INSERT INTO (Add Data in Table)

👉 **INSERT INTO** is used to **add new records (rows) into a table.**

👉 In simple words:

It puts data inside your table.

Basic Syntax

```
SQL
INSERT INTO table_name VALUES (value1, value2, value3);
```

Example

Table: students

id name age

```
SQL
INSERT INTO students VALUES (1, 'Aamir', 22);
```

 Adds one row to the table

Insert Multiple Rows

```
SQL
INSERT INTO students VALUES
(2, 'Rahul', 21),
(3, 'Sara', 23);
```

 Adds multiple records at once

Insert with Column Names (Best Practice 🔥)

```
SQL
INSERT INTO students (id, name, age)
VALUES (4, 'Ali', 20);
```

Why better?

- Order doesn't matter

- Safer when table structure changes

Insert Partial Data

```
SQL
INSERT INTO students (id, name)
VALUES (5, 'John');
```

👉 age will be:

- NULL (if allowed)
- or DEFAULT value

Important Rules

- Values must match column data types
- Follow correct order (if column names not given)
- PRIMARY KEY must be unique
- NOT NULL columns must have value

Interview Answer

👉 *"INSERT INTO is used to add new records into a table. It can insert single or multiple rows and can also specify column names for better flexibility."*

Next Step

Now you should practice:

- 👉 INSERT + SELECT
- 👉 UPDATE & DELETE
- 👉 Constraints with INSERT

If you want:

- ✅ Practice questions
- ✅ Real data examples
- ✅ SQL exercises

Just say: **"Give INSERT practice"** 

You said:

select comaand

ChatGPT said:

 SELECT Command in SQL

👉 **SELECT** is used to **retrieve (fetch) data from a table**.

👉 In simple words:

SELECT = get data from database

 Basic Syntax

```
SQL
SELECT column1, column2
FROM table_name;
```

 Examples

1 Get All Data

```
SQL
SELECT * FROM students;
```

👉 * means **all columns**

2 Get Specific Columns

```
SQL
SELECT name, age FROM students;
```

👉 Only selected columns will be shown

3 Use WHERE (Filter Data)

```
SQL
SELECT * FROM students WHERE age > 20;
```

👉 Returns only rows where condition is true

4 **DISTINCT (Unique Values)**

```
SQL
SELECT DISTINCT age FROM students;
```

👉 Removes duplicate values

5 **ORDER BY (Sorting)**

```
SQL
SELECT * FROM students ORDER BY age DESC;
```

👉 DESC = highest first

👉 ASC = lowest first

6 **LIMIT (Top Records)**

```
SQL
SELECT * FROM students LIMIT 2;
```

👉 Shows only first 2 rows

7 **COUNT (Aggregate Function)**

```
SQL
SELECT COUNT(*) FROM students;
```

👉 Counts total rows

📌 **Combined Example (Important 🔥)**

```
SQL
SELECT name, age
FROM students
WHERE age > 20
ORDER BY age DESC
LIMIT 3;
```

Meaning:

- Get name & age
- Only age > 20
- Sort descending
- Show top 3

Interview Answer

👉 *"SELECT is used to retrieve data from a database. It can be combined with clauses like WHERE, ORDER BY, DISTINCT, and LIMIT to filter and organize the results."*

Next Step

Now you should practice:

- 👉 SELECT + WHERE 🔥
- 👉 GROUP BY + HAVING
- 👉 JOIN (most important)

If you want:

- ✅ 20 SELECT practice questions
- ✅ Real interview queries
- ✅ Mini project

Just say: **"Give SELECT practice"** 💯

You said:

our first database Great—let's build your first database from scratch step by step (simple but real-world) 🚀 📌 Step 1: Create Database CREATE DATABASE first_db; USE first_db; 👉 Creates and selects your database 📌 Step 2: Create Tables 👤 Users Table CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(50) NOT NULL, email VARCHAR(100) UNIQUE, age INT CHECK (age >= 18)); 📦 Orders Table (with Foreign Key) CREATE TABLE orders (order_id INT PRIMARY KEY, product VARCHAR(50), user_id INT, FOREIGN KEY (user_id) REFERENCES users(id)); 👉 This creates a relationship between users and orders 📌 Step 3: Insert Data INSERT INTO users VALUES (1, 'Aamir', 'aamir@gmail.com', 22), (2, 'Rahul', 'rahul@gmail.com', 21), (3, 'Sara', 'sara@gmail.com', 23); INSERT INTO orders VALUES (101, 'Laptop', 1), (102,

'Phone', 2), (103, 'Mouse', 1); ✨ Step 4: View Data `SELECT * FROM users;`
`SELECT * FROM orders;` ✨ Step 5: JOIN (Combine Data 🔥) `SELECT u.name, o.product FROM users u JOIN orders o ON u.id = o.user_id;` 🧠 Output name product Aamir Laptop Rahul Phone Aamir Mouse ✨ Step 6: Filter Data `SELECT * FROM users WHERE age > 21;` ✨ Step 7: Update Data `UPDATE users SET age = 24 WHERE id = 1;` ✨ Step 8: Delete Data `DELETE FROM users WHERE id = 3;`
 🎯 What You Learned ✓ Create database ✓ Create tables ✓ Primary key & foreign key ✓ Insert data ✓ SELECT & WHERE ✓ JOIN (very important 🔥) ✓ UPDATE & DELETE ¹⁰⁰ Final Interview Line 👉 "In my first database project, I created tables with primary and foreign keys, inserted data, and used SQL queries like SELECT, JOIN, UPDATE, and DELETE to manage and retrieve data."
 🚀 Next Step (Important) Now you're ready for: 👉 Mini Project (Instagram Database 🔥) 👉 Banking System Database 👉 Advanced JOIN + Queries Just say: 👉 "Start Instagram project" or 👉 "Give next level project" 🚀 our first table Nice—now let's build your first table step by step (super simple + clear)
 🚀 ✨ Step 1: Create Database `CREATE DATABASE first_db; USE first_db;` ✨ Step 2: Create Your First Table `CREATE TABLE students ( id INT PRIMARY KEY, name VARCHAR(50) NOT NULL, age INT );` 🧠 Explanation id → unique identifier (Primary Key) name → cannot be empty (NOT NULL) age → normal column 👉 This table will store student data ✨ Step 3: Insert Data `INSERT INTO students VALUES (1, 'Aamir', 22), (2, 'Rahul', 21), (3, 'Sara', 23);` ✨ Step 4: View Table Data `SELECT * FROM students;` 📊 Output id name age 1 Aamir 22 2 Rahul 21 3 Sara 23 ✨ Step 5: Simple Queries 👉 Filter Data `SELECT * FROM students WHERE age > 21;` 👉 Select Specific Columns `SELECT name, age FROM students;` 🎯 What You Learned ✓ Create table ✓ Add data (INSERT) ✓ View data (SELECT) ✓ Filter data (WHERE) ¹⁰⁰ Interview Line 👉 "A table is a collection of data organized in rows and columns. I created my first table using CREATE TABLE, inserted data using INSERT, and retrieved data using SELECT."
 🚀 Next Step Now you should learn: 👉 Constraints (NOT NULL, UNIQUE, DEFAULT) 👉 Keys (Primary + Foreign) 👉 Joins (very important 🔥) Just say: 👉 "Next level table" or 👉 "Give practice questions on table" ¹⁰⁰ database query ✨ What is a Database Query? 👉 A database query is a command used to interact with a database. 👉 In simple words: Query = asking the database to do something (get, add, update, delete data) ✨ Types of Database Queries **1** SELECT (Read Data) 👉 Used to fetch data `SELECT * FROM students;` ✓ Gets all data `SELECT name, age FROM students;` ✓ Gets specific columns **2** INSERT (Add Data) 👉 Used to insert new records `INSERT INTO students VALUES (4, 'Ali', 20);` **3** UPDATE (Modify Data) 👉 Used to change existing data `UPDATE students SET age = 21 WHERE id = 4;` **4** DELETE (Remove Data) 👉 Used to delete records `DELETE FROM students WHERE id = 4;` **5** CREATE (Create Table/Database) `CREATE TABLE students ( id INT, name VARCHAR(50) );` **6** DROP (Delete Table/Database) `DROP TABLE students;` **7**

ALTER (Modify Table) ALTER TABLE students ADD email VARCHAR(50); ✨

Important Query Clauses 👉 WHERE (Filter) SELECT * FROM students WHERE age > 20; 👉 ORDER BY (Sort) SELECT * FROM students ORDER BY age DESC; 👉 GROUP BY (Group Data) SELECT age, COUNT(*) FROM students GROUP BY age; 👉 LIMIT (Restrict Results) SELECT * FROM students LIMIT 2; 🎯 Final Interview Answer 👉 "A database query is a command used to interact with a database to retrieve, insert, update, or delete data. Common queries include SELECT, INSERT, UPDATE, and DELETE." 🚀 Next Step To become strong in SQL: 👉 Practice: SELECT + WHERE 🔥 GROUP BY + HAVING JOIN (most important)

If you want: ✅ Practice questions ✅ Real interview queries ✅ Mini project

Just say: "Give query practice" 100 create table ✨ CREATE TABLE in SQL 👉

CREATE TABLE is used to create a new table in a database. 👉 In simple words: It defines structure (columns + data types) where data will be stored. ✨ Basic Syntax CREATE TABLE table_name (column1 datatype, column2 datatype, column3 datatype); ✨ Example (Your First Table) CREATE TABLE students (id INT, name VARCHAR(50), age INT); 🗨 Explanation id → number name → text (max 50 characters) age → number ✨ With Constraints (Important 🔥)

CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(50) NOT NULL, email VARCHAR(100) UNIQUE, age INT CHECK (age >= 18), salary INT DEFAULT 25000); 🗨 What's happening: PRIMARY KEY → unique + not null NOT NULL → cannot be empty UNIQUE → no duplicate values CHECK → condition applied DEFAULT → auto value if not given ✨ Create Table with Foreign Key CREATE TABLE orders (order_id INT PRIMARY KEY, product VARCHAR(50), user_id INT, FOREIGN KEY (user_id) REFERENCES students(id)); 👉 Connects two tables (relationship) ✨ Check Table SHOW TABLES; DESCRIBE students; 🎯 Interview Answer 👉 "CREATE TABLE is used to define a new table in a database by specifying column names, data types, and constraints like primary key and foreign key." 🚀 Next Step Now you should learn: 👉 INSERT (add data) 👉 SELECT (get data) 👉 WHERE + JOIN 🔥 If you want: ✅ Practice questions on CREATE TABLE ✅ Real project (Instagram DB) ✅ Advanced schema design

Just say: "Next step INSERT" 100 what are constraints ✨ What are Constraints in SQL? 👉 Constraints are rules applied to table columns to control what data can be stored. 👉 In simple words: They ensure the data is correct, valid, and consistent. ✨ Why Constraints are Important? Prevent wrong data ✖

Maintain data accuracy ✅ Improve data integrity 🔥 ✨ Types of Constraints

- 1 NOT NULL 👉 Column cannot have empty (NULL) values name VARCHAR(50) NOT NULL
- 2 UNIQUE 👉 All values must be different email VARCHAR(50) UNIQUE
- 3 PRIMARY KEY 🔥 👉 Combination of: UNIQUE ✅ NOT NULL ✅ id INT PRIMARY KEY
- 4 FOREIGN KEY 🔥 👉 Used to connect two tables FOREIGN KEY (user_id) REFERENCES users(id)
- 5 DEFAULT 👉 Sets a default value salary INT DEFAULT 25000
- 6 CHECK 👉 Restricts values using a condition CHECK (age >= 18) ✨ Full Example CREATE TABLE employees (id

INT PRIMARY KEY, name VARCHAR(50) NOT NULL, email VARCHAR(100) UNIQUE, age INT CHECK (age >= 18), salary INT DEFAULT 25000); 🎯 Final Interview Answer 🙌 "Constraints are rules applied to table columns to ensure data accuracy and integrity. Examples include NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, and DEFAULT." 🚀 Next Step Now you should practice: 🙌 Writing tables with constraints 🔥 🙌 Using PRIMARY KEY & FOREIGN KEY 🙌 JOIN queries If you want: ✅ Practice questions ✅ Mini project ✅ Interview questions Just say: "Give constraint practice" 📌 key constraints 📌 Key Constraints in SQL 🙌 Key constraints are rules used to identify records uniquely and maintain relationships between tables. 🙌 In simple words: They help keep data unique and connected correctly. 🔑 Main Key Constraints

1 PRIMARY KEY 🔥 🙌 A Primary Key: Uniquely identifies each row Cannot be NULL ❌ Cannot have duplicates ❌ Only one per table ✅ Example CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(50)); 🗨️ Meaning Each student has a unique id No two rows can have same id

2 FOREIGN KEY 🔥 🙌 A Foreign Key: Connects two tables Refers to Primary Key of another table Can have duplicates ✅ Can have NULL values ✅

Example CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(50)); CREATE TABLE orders (order_id INT PRIMARY KEY, student_id INT, FOREIGN KEY (student_id) REFERENCES students(id)); 🗨️ Meaning student_id must exist in students.id Maintains relationship 📌 Other Important Keys (Short)

3 UNIQUE KEY 🙌 Ensures all values are different email VARCHAR(100) UNIQUE

4 COMPOSITE KEY 🙌 Combination of multiple columns as key PRIMARY KEY (id, email)

5 CANDIDATE KEY 🙌 Possible columns that can be primary key

📌 Comparison (Important 🔥) Feature Primary Key Foreign Key Purpose Unique ID Link tables NULL allowed ❌ No ✅ Yes Duplicate allowed ❌ No ✅ Yes Count One Many 🎯 Interview Answer 🙌 "Key constraints are used to uniquely identify records and maintain relationships between tables. The primary key uniquely identifies each row, while the foreign key links tables by referencing a primary key." 🚀 Next Step Now you should learn: 🙌 JOIN using keys (most important 🔥) 🙌 Real database design 🙌 Advanced queries If you want: ✅ JOIN practice questions ✅ Real project (Instagram DB) ✅ Mock interview Just say: "Give JOIN practice" 📌 primary and foreign keys

📌 Primary Key vs Foreign Key (Simple + Clear) 🔑 1. Primary Key (PK) 🙌 A Primary Key is a column (or columns) that: Uniquely identifies each row Cannot be NULL Cannot have duplicate values Only one primary key per table ✅ Example CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(50)); 🗨️ Example Data id name 1 Aamir 2 Rahul ✔️ Each id is unique ❌ Duplicate or NULL → not allowed 🔗 2. Foreign Key (FK) 🙌 A Foreign Key is a column that: Creates a link between two tables Refers to Primary Key of another table Can have duplicate values Can have NULL values ✅ Example CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(50)); CREATE TABLE orders (

order_id INT PRIMARY KEY, student_id INT, FOREIGN KEY (student_id) REFERENCES students(id)); 🧠 Example Data students id name 1 Aamir 2 Rahul

orders order_id student_id 101 1 102 1 103 NULL 104 5 ⚡ Key Differences (Interview 🔥) Feature Primary Key Foreign Key Purpose Unique ID Link tables NULL allowed ❌ No ✅ Yes Duplicate allowed ❌ No ✅ Yes Count per table One Multiple 🎯 Final Interview Answer 👉 "A primary key uniquely identifies each record in a table and cannot be null or duplicate. A foreign key is used to create relationships between tables by referencing the primary key of another table." 🚀 Next Step To fully understand keys, you should now learn: 👉 JOIN using primary & foreign key (🔥 most important) 👉 Real database design (projects) If you want: ✅ 10 JOIN practice questions ✅ Mini project (Instagram DB) ✅ Mock interview Just say: "Give JOIN questions" 100% insert into table 🌟 INSERT INTO (Add Data in Table) 👉 INSERT INTO is used to add new records (rows) into a table. 👉 In simple words: It puts data inside your table. 🌟 Basic Syntax INSERT INTO table_name VALUES (value1, value2, value3); 🌟 Example 🗨 Table: students id name age INSERT INTO students VALUES (1, 'Aamir', 22); 👉 Adds one row to the table 🌟 Insert Multiple Rows INSERT INTO students VALUES (2, 'Rahul', 21), (3, 'Sara', 23); 👉 Adds multiple records at once 🌟 Insert with Column Names (Best Practice 🔥) INSERT INTO students (id, name, age) VALUES (4, 'Ali', 20); 🧠 Why better? Order doesn't matter Safer when table structure changes 🌟 Insert Partial Data INSERT INTO students (id, name) VALUES (5, 'John'); 👉 age will be: NULL (if allowed) or DEFAULT value 🌟 Important Rules ⚠ Values must match column data types Follow correct order (if column names not given) PRIMARY KEY must be unique NOT NULL columns must have value 🎯 Interview Answer 👉 "INSERT INTO is used to add new records into a table. It can insert single or multiple rows and can also specify column names for better flexibility." 🚀 Next Step Now you should practice: 👉 INSERT + SELECT 👉 UPDATE & DELETE 👉 Constraints with INSERT If you want: ✅ Practice questions ✅ Real data examples ✅ SQL exercises Just say: "Give INSERT practice" 100% select command 🌟 SELECT Command in SQL 👉 SELECT is used to retrieve (fetch) data from a table. 👉 In simple words: SELECT = get data from database 🌟 Basic Syntax SELECT column1, column2 FROM table_name; 🌟 Examples 1 Get All Data SELECT * FROM students; 👉 * means all columns 2 Get Specific Columns SELECT name, age FROM students; 👉 Only selected columns will be shown 3 Use WHERE (Filter Data) SELECT * FROM students WHERE age > 20; 👉 Returns only rows where condition is true 4 DISTINCT (Unique Values) SELECT DISTINCT age FROM students; 👉 Removes duplicate values 5 ORDER BY (Sorting) SELECT * FROM students ORDER BY age DESC; 👉 DESC = highest first 👉 ASC = lowest first 6 LIMIT (Top Records) SELECT * FROM students LIMIT 2; 👉 Shows only first 2 rows 7 COUNT (Aggregate Function) SELECT COUNT(*) FROM students; 👉 Counts total rows 🌟 Combined Example

(Important 🔥) `SELECT name, age FROM students WHERE age > 20 ORDER BY age DESC LIMIT 3;` 🧠 Meaning: Get name & age Only age > 20 Sort descending Show top 3 🎯 Interview Answer 👉 "SELECT is used to retrieve data from a database. It can be combined with clauses like WHERE, ORDER BY, DISTINCT, and LIMIT to filter and organize the results." 🚀 Next Step Now you should practice: 👉 SELECT + WHERE 🔥 👉 GROUP BY + HAVING 👉 JOIN (most important) If you want: ✅ 20 SELECT practice questions ✅ Real interview queries ✅ Mini project Just say: "Give SELECT practice" 💯 explain all practically wise

ChatGPT said:

You've collected a lot of concepts 👉 — now let's **connect everything practically (real-world thinking)** so you understand *how SQL is actually used in projects*, not just theory.

🚀 REAL-WORLD PRACTICAL FLOW (Step by Step)

Think:

👉 You are building a **simple app (like Instagram / Shopping app)**

📌 1. CREATE DATABASE (Start Project)

```
SQL
CREATE DATABASE app_db;
USE app_db;
```

👉 Practical meaning:

You create a **space to store all app data**

📌 2. CREATE TABLE (Design Structure)

```
SQL
CREATE TABLE users (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  age INT CHECK (age >= 18)
);
```

👉 Practical thinking:

- **id** → unique user ID
- **name** → required field
- **email** → must be unique
- **age** → only valid users

📌 3. CREATE RELATION (FOREIGN KEY)

```
SQL
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    product VARCHAR(50),
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

👉 Practical thinking:

- One user → many orders
- Connect tables using **foreign key**

📌 4. INSERT (Add Real Data)

```
SQL
INSERT INTO users VALUES
(1, 'Aamir', 'aamir@gmail.com', 22),
(2, 'Rahul', 'rahul@gmail.com', 21);
```

👉 Real meaning:

Users sign up → data stored

```
SQL
INSERT INTO orders VALUES
(101, 'Laptop', 1),
(102, 'Phone', 2),
(103, 'Mouse', 1);
```

👉 Real meaning:

Users place orders

5. SELECT (Fetch Data)

```
SQL
SELECT * FROM users;
```

👉 Show all users

```
SQL
SELECT name, age FROM users;
```

👉 Show only needed data

6. WHERE (Filter Data)

```
SQL
SELECT * FROM users WHERE age > 21;
```

👉 Real use:

- Find adult users
- Apply conditions

7. JOIN (🔥 MOST IMPORTANT)

```
SQL
SELECT u.name, o.product
FROM users u
JOIN orders o
ON u.id = o.user_id;
```

👉 Practical meaning:

- Show **which user bought what**

Output:

name	product
------	---------

Aamir	Laptop
-------	--------

name	product
Rahul	Phone
Aamir	Mouse

8. UPDATE (Modify Data)

```
SQL
UPDATE users
SET age = 24
WHERE id = 1;
```

 Real use:

- User updates profile

9. DELETE (Remove Data)

```
SQL
DELETE FROM users WHERE id = 2;
```

 Real use:

- User deletes account

10. CONSTRAINTS (Data Safety)

 Practical role:

- NOT NULL → required fields
- UNIQUE → no duplicate emails
- CHECK → valid age
- PRIMARY KEY → unique ID

- FOREIGN KEY → valid relationships

11. QUERY CLAUSES (Real Usage)

ORDER BY

```
SQL
SELECT * FROM users ORDER BY age DESC;
```

 Show top users

GROUP BY

```
SQL
SELECT user_id, COUNT(*)
FROM orders
GROUP BY user_id;
```

 Count orders per user

LIMIT

```
SQL
SELECT * FROM users LIMIT 1;
```

 Show top record

HOW COMPANIES USE THIS (Important)

 Backend systems:

- Store users
- Store transactions
- Connect data using keys
- Fetch data using JOIN

 Data Analytics:

- Use SELECT + GROUP BY

- Analyze patterns

FINAL INTERVIEW STORY (Very Powerful)

👉 *"I understand SQL practically by designing a small system where I created tables with constraints, inserted real-world data, used SELECT and WHERE for filtering, JOIN for combining tables, and UPDATE/DELETE for managing records."*

WHAT YOU SHOULD DO NEXT

Now don't just read — **practice like this:**

- 👉 Step 1: Create your own database
- 👉 Step 2: Insert 10–20 records
- 👉 Step 3: Write queries:

- SELECT
- WHERE
- JOIN
- GROUP BY

NEXT LEVEL (Highly Recommended)

I can take you to real developer level:

- ✅ Build **Instagram Database Project (step-by-step)**
- ✅ Give **Real SQL assignments (company level)**
- ✅ Do **Mock interview with you**

Just say:

- 👉 **"Start Instagram project"** (best next step) 

You said:

Start Instagram project

ChatGPT said:

Perfect—now we'll build a **mini Instagram database project** step by step like a real developer 

INSTAGRAM DATABASE PROJECT (STEP-BY-STEP)

Step 1: Create Database

```
SQL
CREATE DATABASE instagram_db;
USE instagram_db;
```

 This is your project workspace

Step 2: Create Tables

1. Users Table

```
SQL
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INT CHECK (age >= 13),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

 Stores user accounts

2. Posts Table

```
SQL
CREATE TABLE posts (
    post_id INT PRIMARY KEY,
    image_url VARCHAR(255),
    caption VARCHAR(255),
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

 One user → many posts

♥ 3. Likes Table

SQL

```
CREATE TABLE likes (  
    like_id INT PRIMARY KEY,  
    user_id INT,  
    post_id INT,  
    FOREIGN KEY (user_id) REFERENCES users(user_id),  
    FOREIGN KEY (post_id) REFERENCES posts(post_id)  
);
```

👉 Users like posts

💬 4. Comments Table

SQL

```
CREATE TABLE comments (  
    comment_id INT PRIMARY KEY,  
    comment_text VARCHAR(255),  
    user_id INT,  
    post_id INT,  
    FOREIGN KEY (user_id) REFERENCES users(user_id),  
    FOREIGN KEY (post_id) REFERENCES posts(post_id)  
);
```

👉 Users comment on posts

📌 Step 3: Insert Sample Data

👤 Users

SQL

```
INSERT INTO users VALUES  
(1, 'aamir', 'aamir@gmail.com', 22, NOW()),  
(2, 'rahul', 'rahul@gmail.com', 21, NOW()),  
(3, 'sara', 'sara@gmail.com', 23, NOW());
```

Posts

```
SQL
INSERT INTO posts VALUES
(101, 'img1.jpg', 'My first post', 1),
(102, 'img2.jpg', 'Hello world', 2),
(103, 'img3.jpg', 'Enjoying life', 1);
```

Likes

```
SQL
INSERT INTO likes VALUES
(1, 2, 101),
(2, 3, 101),
(3, 1, 102);
```

Comments

```
SQL
INSERT INTO comments VALUES
(1, 'Nice post!', 2, 101),
(2, 'Awesome!', 3, 101),
(3, 'Great!', 1, 102);
```

Step 4: Important Queries (Real Use)

1. Show all posts with user names

```
SQL
SELECT u.username, p.caption
FROM users u
JOIN posts p ON u.user_id = p.user_id;
```

2. Count likes on each post

```
SQL
SELECT post_id, COUNT(*) AS total_likes
FROM likes
GROUP BY post_id;
```

3. Show comments with usernames

```
SQL
SELECT u.username, c.comment_text
FROM comments c
JOIN users u ON c.user_id = u.user_id;
```

4. Find most liked post

```
SQL
SELECT post_id, COUNT(*) AS likes
FROM likes
GROUP BY post_id
ORDER BY likes DESC
LIMIT 1;
```

5. Users who posted more than 1 post

```
SQL
SELECT user_id, COUNT(*)
FROM posts
GROUP BY user_id
HAVING COUNT(*) > 1;
```

What You Built (Real Skills)

- ✓ Database design
- ✓ Multiple tables
- ✓ Primary & Foreign keys
- ✓ Real relationships
- ✓ JOIN queries
- ✓ GROUP BY & analytics

Interview Explanation (VERY IMPORTANT)

 *"I built an Instagram-like database with users, posts, likes, and comments tables. I used primary and foreign keys to maintain relationships and wrote queries using JOIN, GROUP BY, and aggregate functions to fetch insights like most liked posts and user activity."*

NEXT LEVEL (Choose One)

Now you're ready for:

- 👉 🔥 **Advanced Queries on this project**
- 👉 🏠 **Banking System Database Project**
- 👉 🎯 **Mock Interview (SQL + Project Explanation)**

Just say:

- 👉 **"Give advanced queries"** or
- 👉 **"Start banking project"** or
- 👉 **"Take mock interview"** ¹⁰⁰

You said:

Give Real SQL assignments (company level)

ChatGPT said:

Good—this is where you actually become strong ¹⁰⁰

Below are **real company-level SQL assignments** based on your **Instagram database project** (same schema you created).

REAL SQL ASSIGNMENTS (COMPANY LEVEL)

👉 Tables you have:

- **users**
- **posts**
- **likes**
- **comments**

LEVEL 1 (Warm-up but Important)

1 Find all users who are older than 21

```
SQL
SELECT * FROM users WHERE age > 21;
```

2 Get all posts created by user_id = 1

```
SQL
SELECT * FROM posts WHERE user_id = 1;
```

3 Show usernames and their posts

```
SQL
SELECT u.username, p.caption
FROM users u
JOIN posts p ON u.user_id = p.user_id;
```

4 Count total users

```
SQL
SELECT COUNT(*) FROM users;
```

5 List all unique ages

```
SQL
SELECT DISTINCT age FROM users;
```

LEVEL 2 (Real Interview Level)

6 Count number of posts per user

```
SQL
SELECT user_id, COUNT(*) AS total_posts
FROM posts
GROUP BY user_id;
```

7 Find users who have posted more than 1 post

```
SQL
SELECT user_id, COUNT(*)
FROM posts
GROUP BY user_id
HAVING COUNT(*) > 1;
```

8 Count likes on each post

```
SQL
SELECT post_id, COUNT(*) AS total_likes
FROM likes
GROUP BY post_id;
```

9 Find posts with more than 1 like

```
SQL
SELECT post_id, COUNT(*)
FROM likes
GROUP BY post_id
HAVING COUNT(*) > 1;
```

10 Show comments with usernames

```
SQL
SELECT u.username, c.comment_text
FROM comments c
JOIN users u ON c.user_id = u.user_id;
```

LEVEL 3 (Advanced / Company Level)

1 1 Find the most liked post

```
SQL
SELECT post_id, COUNT(*) AS likes
FROM likes
GROUP BY post_id
ORDER BY likes DESC
LIMIT 1;
```

1 2 Find user who got most likes (🔥 very important)

```
SQL
SELECT p.user_id, COUNT(*) AS total_likes
FROM posts p
JOIN likes l ON p.post_id = l.post_id
GROUP BY p.user_id
ORDER BY total_likes DESC
LIMIT 1;
```

1 3 Find users who never posted anything

```
SQL
SELECT u.user_id, u.username
FROM users u
LEFT JOIN posts p ON u.user_id = p.user_id
WHERE p.post_id IS NULL;
```

1 4 Find posts with no comments

```
SQL
SELECT p.post_id
FROM posts p
LEFT JOIN comments c ON p.post_id = c.post_id
WHERE c.comment_id IS NULL;
```

1 5 Find most active user (posts + comments)

```
SQL
SELECT u.user_id,
       COUNT(DISTINCT p.post_id) + COUNT(DISTINCT
c.comment_id) AS activity
FROM users u
LEFT JOIN posts p ON u.user_id = p.user_id
LEFT JOIN comments c ON u.user_id = c.user_id
GROUP BY u.user_id
ORDER BY activity DESC
LIMIT 1;
```

LEVEL 4 (Hard / FAANG Style)

1 6 Find users who liked their own posts

```
SQL
SELECT DISTINCT u.username
FROM users u
JOIN posts p ON u.user_id = p.user_id
JOIN likes l ON p.post_id = l.post_id
WHERE u.user_id = l.user_id;
```

1 7 Find top 2 most liked posts

```
SQL
SELECT post_id, COUNT(*) AS likes
FROM likes
GROUP BY post_id
ORDER BY likes DESC
LIMIT 2;
```

1 8 Find users who never liked any post

```
SQL
SELECT u.user_id, u.username
FROM users u
LEFT JOIN likes l ON u.user_id = l.user_id
WHERE l.like_id IS NULL;
```

1 9 Find average number of likes per post

```
SQL
SELECT AVG(like_count)
FROM (
    SELECT COUNT(*) AS like_count
    FROM likes
    GROUP BY post_id
) AS temp;
```


2 0 Find users who commented on every post

```
SQL
SELECT user_id
FROM comments
GROUP BY user_id
HAVING COUNT(DISTINCT post_id) = (SELECT COUNT(*) FROM posts);
```

What Companies Test Here

👉 You just practiced:

- JOIN 🔥
- GROUP BY + HAVING 🔥
- Subqueries 🔥
- Real-world logic

Interview Tip

👉 If you can solve:

- Q12 (most liked user)
- Q13 (no posts users)
- Q16 (self-like users)

👉 You are already **above average candidate**

Next Level (Very Important)

Now don't stop here. I can take you further:

- ✓ 50 FAANG-level SQL questions
- ✓ Live mock interview (like real company)
- ✓ Advanced topics (Window Functions 🔥)

Just say:

- 👉 "Take mock interview" or
- 👉 "Give FAANG SQL questions" 🚀

You said:

Give FAANG SQL questions

ChatGPT said:

Alright—now we're stepping into **FAANG-level SQL** 🚀

These are **real-style questions** asked in companies like **Google, Amazon, Meta, and Netflix**.

👉 I'll give you **high-quality questions + solutions + thinking approach** (this is what actually matters).

🔥 FAANG SQL QUESTIONS (WITH SOLUTIONS)

📌 1. Second Highest Salary

👉 Very famous question

```
SQL
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

🧠 Logic:

- Find highest salary
- Then find max below it

📌 2. Nth Highest Salary (Generic 🔥)

```
SQL
SELECT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

👉 3rd highest salary

3. Find Duplicate Records

```
SQL
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

4. Employees Without Manager

```
SQL
SELECT *
FROM employees
WHERE manager_id IS NULL;
```

5. Find Customers Who Never Ordered

```
SQL
SELECT c.customer_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

6. Top 3 Salaries Per Department 🔥

```
SQL
SELECT *
FROM (
    SELECT e.*,
           DENSE_RANK() OVER (PARTITION BY department_id ORDER
BY salary DESC) AS rnk
    FROM employees e
) t
WHERE rnk <= 3;
```

Concept:

Window Function (VERY IMPORTANT)

7. Running Total (Cumulative Sum)

```
SQL
SELECT id,
       salary,
       SUM(salary) OVER (ORDER BY id) AS running_total
FROM employees;
```

8. Find Consecutive Days Login

```
SQL
SELECT user_id
FROM logins
GROUP BY user_id
HAVING COUNT(DISTINCT login_date) >= 3;
```

9. Find Most Active User

```
SQL
SELECT user_id, COUNT(*) AS activity
FROM actions
GROUP BY user_id
ORDER BY activity DESC
LIMIT 1;
```

10. Find Employees Earning More Than Their Manager

```
SQL
SELECT e.name
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

11. Find Missing Numbers

```
SQL
SELECT t1.id + 1 AS missing_id
FROM numbers t1
LEFT JOIN numbers t2 ON t1.id + 1 = t2.id
WHERE t2.id IS NULL;
```

12. Find First Order Per Customer

```
SQL
SELECT customer_id, MIN(order_date)
FROM orders
GROUP BY customer_id;
```

13. Find Highest Salary in Each Department

```
SQL
SELECT department_id, MAX(salary)
FROM employees
GROUP BY department_id;
```

14. Delete Duplicate Rows

```
SQL
DELETE FROM users
WHERE id NOT IN (
    SELECT MIN(id)
    FROM users
    GROUP BY email
);
```

15. Find Users Who Logged in Yesterday but Not Today

```
SQL
SELECT user_id
FROM logins
WHERE login_date = CURDATE() - INTERVAL 1 DAY
AND user_id NOT IN (
    SELECT user_id FROM logins WHERE login_date = CURDATE()
);
```

16. Find Median Salary 🔥

```
SQL
SELECT AVG(salary) AS median
FROM (
    SELECT salary
    FROM employees
    ORDER BY salary
    LIMIT 2 - (SELECT COUNT(*) FROM employees) % 2
    OFFSET (SELECT (COUNT(*) - 1) / 2 FROM employees)
) x;
```

17. Find Users Who Liked Their Own Posts

```
SQL
SELECT DISTINCT u.user_id
FROM users u
JOIN posts p ON u.user_id = p.user_id
JOIN likes l ON p.post_id = l.post_id
WHERE u.user_id = l.user_id;
```

18. Find Top Product by Sales

```
SQL
SELECT product_id, SUM(amount) AS total_sales
FROM sales
GROUP BY product_id
ORDER BY total_sales DESC
LIMIT 1;
```

19. Find Percentage of Each Category

```
SQL
SELECT category,
       COUNT(*) * 100.0 / (SELECT COUNT(*) FROM products) AS
percentage
FROM products
GROUP BY category;
```

20. Find Gap Between Dates

```
SQL
SELECT user_id,
       DATEDIFF(MAX(login_date), MIN(login_date)) AS gap
FROM logins
GROUP BY user_id;
```

What FAANG Actually Tests

 These questions test:

- JOIN 
- GROUP BY + HAVING 
- Subqueries 
- Window Functions  (VERY IMPORTANT)
- Real-world thinking

Honest Reality Check

If you can solve:

- Q6 (Top 3 per dept)
- Q10 (salary vs manager)
- Q16 (median)

 You are already **strong candidate**

NEXT LEVEL (GAME CHANGER)

Now you're ready for:

- 👉 🔥 **Window Functions Deep Dive**
- 👉 🎯 **Live Mock Interview (FAANG style)**
- 👉 📊 **Data Analytics SQL Case Study**

Just say:

- 👉 **"Take FAANG mock interview"** (best next step) 🚀

You said:

- ✅ 50 FAANG-level SQL questions

ChatGPT said:

Alright—here are **50 FAANG-level SQL questions** 🔥

No fluff. These are the kinds of problems asked at companies like Google, Amazon, Meta, and Netflix.

- 👉 I've grouped them by difficulty so you can **progress like a pro**.

LEVEL 1 (Core Foundation – Must Master)

1. Find second highest salary
2. Find Nth highest salary
3. Find duplicate records in a table
4. Remove duplicate rows
5. Find employees with NULL manager
6. Count total number of employees
7. List all unique values in a column
8. Find records with maximum value
9. Find records with minimum value

10. Count number of rows per group

LEVEL 2 (Intermediate – Real Interview Level)

11. Find customers who never placed an order

12. Find customers who placed at least one order

13. Count number of orders per customer

14. Find top 3 highest salaries

15. Find department-wise highest salary

16. Find department-wise average salary

17. Find employees earning more than average

18. Find employees hired in last 30 days

19. Find total sales per product

20. Find top selling product

LEVEL 3 (JOIN + GROUP BY Mastery)

21. Show employees with their department names

22. Find employees working in same department

23. Find customers and their orders

24. Find products never sold

25. Find orders without customers

26. Find users with no activity

27. Find posts with no likes

28. Count likes per post

29. Count comments per post

30. Find most active user

LEVEL 4 (Advanced Logic – FAANG Level)

31. Find top 3 salaries per department 

32. Find employees earning more than their manager 🔥
33. Find consecutive login days
34. Find users who logged in every day
35. Find gap between two dates
36. Find running total of sales
37. Find moving average
38. Find rank of each employee by salary
39. Find dense rank of each employee
40. Find duplicate emails but keep latest record

🔥 LEVEL 5 (Hard / Real FAANG Questions)

41. Find median salary 🔥
42. Find percentage contribution of each category
43. Find churned users (inactive users)
44. Find cohort retention
45. Find first and last transaction per user
46. Find users who liked their own posts
47. Find users who never liked any post
48. Find most liked post per user
49. Find users with maximum activity (posts + comments)
50. Find recommendation system logic (users who liked same posts)

🎯 How to Use This List (Very Important)

Don't try to memorize ❌

Do this instead:

- 👉 Step 1: Solve Level 1 + 2
- 👉 Step 2: Practice JOIN + GROUP BY
- 👉 Step 3: Master Window Functions (Level 4)
- 👉 Step 4: Try Level 5 (real FAANG)

Reality Check

If you can confidently solve:

- Top 3 per department
- Salary vs manager
- Running total
- Median

 You are already **top 10–20% SQL candidate**

Next Step (Best Move)

I can now help you with:

- ✓ Detailed **solutions for all 50 questions**
- ✓ **Step-by-step explanations (beginner → advanced)**
- ✓ **Mock FAANG SQL interview**

Just say:

-  **"Give all 50 solutions"** or
-  **"Take mock interview"** 

You said:

Give all 50 solutions

ChatGPT said:

Alright—here are **solutions for all 50 FAANG-level SQL questions** 
I'll keep them **clean, practical, and interview-ready** (no unnecessary theory).

LEVEL 1 (Solutions)

1. Second Highest Salary

```
SQL
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

2. Nth Highest Salary (3rd)

```
SQL
SELECT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

3. Duplicate Records

```
SQL
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

4. Remove Duplicates

```
SQL
DELETE FROM users
WHERE id NOT IN (
    SELECT MIN(id)
    FROM users
    GROUP BY email
);
```

5. Employees with NULL manager

```
SQL
SELECT * FROM employees WHERE manager_id IS NULL;
```

6. Count Employees

```
SQL
SELECT COUNT(*) FROM employees;
```

7. Unique Values

```
SQL
SELECT DISTINCT department FROM employees;
```

8. Max Value

```
SQL
SELECT * FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

9. Min Value

```
SQL
SELECT * FROM employees
WHERE salary = (SELECT MIN(salary) FROM employees);
```

10. Count per Group

```
SQL
SELECT department, COUNT(*)
FROM employees
GROUP BY department;
```

LEVEL 2

11. Customers never ordered

```
SQL
SELECT c.id
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.id IS NULL;
```

12. Customers who ordered

```
SQL
SELECT DISTINCT customer_id FROM orders;
```

13. Orders per customer

```
SQL
SELECT customer_id, COUNT(*)
FROM orders
GROUP BY customer_id;
```

14. Top 3 salaries

```
SQL
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

15. Dept-wise highest salary

```
SQL
SELECT department, MAX(salary)
FROM employees
GROUP BY department;
```

16. Dept-wise average salary

```
SQL
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

17. Above average salary

```
SQL
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

18. Last 30 days employees

```
SQL
SELECT * FROM employees
WHERE hire_date >= CURDATE() - INTERVAL 30 DAY;
```

19. Sales per product

```
SQL
SELECT product_id, SUM(amount)
FROM sales
GROUP BY product_id;
```

20. Top selling product

```
SQL
SELECT product_id, SUM(amount) AS total
FROM sales
GROUP BY product_id
ORDER BY total DESC
LIMIT 1;
```

LEVEL 3

21. Employees with department

```
SQL
SELECT e.name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.id;
```

22. Same department employees

```
SQL
SELECT e1.name, e2.name
FROM employees e1
JOIN employees e2
ON e1.dept_id = e2.dept_id AND e1.id <> e2.id;
```

23. Customers with orders

```
SQL
SELECT c.name, o.id
FROM customers c
JOIN orders o ON c.id = o.customer_id;
```

24. Products never sold

```
SQL
SELECT p.id
FROM products p
LEFT JOIN sales s ON p.id = s.product_id
WHERE s.product_id IS NULL;
```

25. Orders without customers

```
SQL
SELECT o.id
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.id
WHERE c.id IS NULL;
```

26. Users with no activity

```
SQL
SELECT u.id
FROM users u
LEFT JOIN actions a ON u.id = a.user_id
WHERE a.user_id IS NULL;
```

27. Posts with no likes

```
SQL
SELECT p.id
FROM posts p
LEFT JOIN likes l ON p.id = l.post_id
WHERE l.post_id IS NULL;
```

28. Likes per post

```
SQL
SELECT post_id, COUNT(*)
FROM likes
GROUP BY post_id;
```


29. Comments per post

```
SQL
SELECT post_id, COUNT(*)
FROM comments
GROUP BY post_id;
```

30. Most active user

```
SQL
SELECT user_id, COUNT(*) AS activity
FROM actions
GROUP BY user_id
ORDER BY activity DESC
LIMIT 1;
```

🔥 LEVEL 4 (Window Functions 🔥)

31. Top 3 per department

```
SQL
SELECT *
FROM (
    SELECT *,
        DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary
DESC) rnk
    FROM employees
) t
WHERE rnk <= 3;
```

32. Salary > manager

```
SQL
SELECT e.name
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

33. Consecutive logins

```
SQL
SELECT user_id
FROM logins
GROUP BY user_id
HAVING COUNT(DISTINCT login_date) >= 3;
```

34. Daily login users

```
SQL
SELECT user_id
FROM logins
GROUP BY user_id
HAVING COUNT(DISTINCT login_date) = (SELECT COUNT(DISTINCT
login_date) FROM logins);
```

35. Date gap

```
SQL
SELECT user_id, DATEDIFF(MAX(date), MIN(date))
FROM logins
GROUP BY user_id;
```

36. Running total

```
SQL
SELECT id, SUM(amount) OVER (ORDER BY id)
FROM sales;
```

37. Moving average

```
SQL
SELECT id, AVG(amount) OVER (ORDER BY id ROWS BETWEEN 2
PRECEDING AND CURRENT ROW)
FROM sales;
```

38. Rank salary

```
SQL
SELECT name, RANK() OVER (ORDER BY salary DESC)
FROM employees;
```

39. Dense rank

```
SQL
SELECT name, DENSE_RANK() OVER (ORDER BY salary DESC)
FROM employees;
```

40. Keep latest duplicate

```
SQL
DELETE FROM users
WHERE id NOT IN (
    SELECT MAX(id)
    FROM users
    GROUP BY email
);
```

LEVEL 5 (Hard FAANG)

41. Median salary

```
SQL
SELECT AVG(salary)
FROM (
    SELECT salary
    FROM employees
    ORDER BY salary
    LIMIT 2 - (SELECT COUNT(*) % 2 FROM employees)
    OFFSET (SELECT (COUNT(*) - 1) / 2 FROM employees)
) x;
```

42. Percentage category

```
SQL
SELECT category,
COUNT(*) * 100.0 / (SELECT COUNT(*) FROM products)
FROM products
GROUP BY category;
```

43. Churn users

```
SQL
SELECT user_id
FROM logins
GROUP BY user_id
HAVING MAX(login_date) < CURDATE() - INTERVAL 30 DAY;
```

44. Cohort retention (basic)

```
SQL
SELECT signup_date, COUNT(*)
FROM users
GROUP BY signup_date;
```

45. First & last transaction

```
SQL
SELECT user_id, MIN(date), MAX(date)
FROM transactions
GROUP BY user_id;
```

46. Self-like users

```
SQL
SELECT DISTINCT u.id
FROM users u
JOIN posts p ON u.id = p.user_id
JOIN likes l ON p.id = l.post_id
WHERE u.id = l.user_id;
```

47. Users never liked

```
SQL
SELECT u.id
FROM users u
LEFT JOIN likes l ON u.id = l.user_id
WHERE l.user_id IS NULL;
```

48. Most liked post per user

```
SQL
SELECT user_id, post_id
FROM (
    SELECT p.user_id, l.post_id, COUNT(*) cnt,
    RANK() OVER (PARTITION BY p.user_id ORDER BY COUNT(*)
DESC) rnk
    FROM posts p
    JOIN likes l ON p.id = l.post_id
    GROUP BY p.user_id, l.post_id
) t
WHERE rnk = 1;
```

49. Max activity user

```
SQL
SELECT user_id, COUNT(*)
FROM (
    SELECT user_id FROM posts
    UNION ALL
    SELECT user_id FROM comments
) t
GROUP BY user_id
ORDER BY COUNT(*) DESC
LIMIT 1;
```

50. Recommendation system (basic)

```
SQL
SELECT l1.user_id, l2.user_id
FROM likes l1
JOIN likes l2
ON l1.post_id = l2.post_id
AND l1.user_id <> l2.user_id;
```

FINAL REALITY

If you can confidently solve:

- Window functions (31–39)
- Median (41)
- Recommendation (50)

👉 You are **FAANG-ready for SQL rounds**

Next Step (Highly Recommended)

Now don't just read—**practice + simulate interview**

I can:

- ✓ Take **live mock interview (FAANG style)**
- ✓ Give **real case study (data analyst role)**
- ✓ Teach **window functions deeply**

Just say:

👉 **“Take mock interview”** (best next move) **100**

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).